

Multiplayer, Netcode & Live Services

Unity Netcode for GameObjects and Unity Gaming Services — the two-level Presence Lobby + Party Session model, server-authoritative vessel spawning, the main-thread affinity contract, and the design decisions, bugs, and fixes behind a party system hardened toward “unbreakable”.



PROJECT

Cosmic Shore — Froglet Inc.

AUDIENCE

CTO review · engineering reference · technical deep-dive

SCOPE

Party · Presence · Friends · Netcode · UGS

REVISED

June 2026

ORIENTATION

How to read this document

This dossier is written in two passes over the same system, so it serves both a quick executive read and a deep engineering reference without forcing either audience through the other's level of detail.

START HERE

Part I — Curated Overview

The mental model in plain language: the two-session architecture, the player journey, the handful of decisions everything hangs on, and the bugs that taught us the most. Skimmable in ~15 minutes.

GO DEEP

Part II — Comprehensive Deep-Dive

Subsystem by subsystem, with the exact classes, NetworkVariables, RPCs, retry policies, threading rules, the full B-series bug catalogue, the resilience matrix, tests, and diagnostics.

Conventions. `monospace` denotes a file, class, or API. Coloured call-outs flag design decisions, bug → fix stories, key insights, and pitfalls. Status pills use the project legend: Open Investigating Fixed Deferred. Figures are redrawn from the canonical engineering docs under Docs/.

Contents

PART I — THE CURATED OVERVIEW

Executive summary	9
System context	10
The UGS surface	10
Netcode for GameObjects	10
The two-level session model	12
Why separate them	14
How an invite crosses the gap	14
The player journey	15
Key design decisions	17
Five bugs that taught us the most	19
Where it stands	21
What's solid, what's open	21

PART II — THE COMPREHENSIVE DEEP-DIVE

Tech stack & the UGS surface	24
The UGS services, in depth	24
Architecture in layers	26
The orchestration layer	27
The service layer	27
Authentication & bootstrap	28
Single-writer authentication	28
Bootstrap order	29
The presence lobby	30
Join-or-create, and the simultaneous-create race	30
How invites travel — per-player properties	30
Refresh cadence	30
ForceReset — and why it's dangerous	31
The party session & state machine	32
One create surface, idempotent	32
Three retry classifiers	32
The 7-state lifecycle	32
The invite protocol, end to end	34
The payload	36

Dedup guards	36
Accept = a choreographed transition	36
The player & vessel spawn pipeline	39
Player is a NetworkBehaviour	40
The initializer family	40
Two subtleties that caused real bugs	40
Multiplayer game flow	42
Config sync and AI backfill	42
Domain-aggregated scoring	42
The friends system	43
Facade API	43
Rich presence across scene transitions	43
Two ways to add a friend	43
Threading & main-thread affinity	45
The problem	45
Why the obvious UniTask primitives don't fix it	47
The fix	47
The canary	48
SOAP data architecture	49
HostConnectionDataSO — the party data container	49
FriendsDataSO — the friends container	49
The SOAP value types	49
Resilience & the error matrix	51
Recovery actions by failure class	53
The timing constants that make it robust	53
The bug catalogue	54
The instructive fixes	54
The open ones (with reproductions)	55
Testing strategy	56
Automated tests	56
Manual MPPM procedures	56
The per-commit gate	57
Network diagnostics	58
The record	58
Classification rules (first match wins)	58
Two design choices worth noting	59

Design-decisions ledger	60
A few decisions worth their own note	60

APPENDICES — REFERENCE

File & class index	64
Party / presence services — Assets/_Scripts/Controller/Party/	64
Netcode / multiplayer — Assets/_Scripts/Controller/Multiplayer/	65
Cross-cutting & SOAP	65
Canonical engineering docs — Docs/	65
Glossary	66
UGS SDK call map	68
Sessions & Relay	68
Lobby properties (the invite channel)	68
Authentication	68
Friends	68



PART I

The Curated Overview

A fast, narrative tour of the online stack — the mental model, the load-bearing design decisions, and the five most instructive bugs. Readable end-to-end in ~15 minutes; the deep mechanics live in Part II.



PART I · OVERVIEW

Executive summary

Cosmic Shore is a real-time multiplayer space game. Its online stack is built on **Unity Netcode for GameObjects** for in-game replication and **Unity Gaming Services (UGS)** for everything around the match — identity, discovery, party formation, and transport. This document explains how those pieces fit together, the decisions that shaped them, and the bugs we fought to make party play feel seamless.

The defining idea is a **two-level session model**. A lobby-only **Presence Lobby** lets every signed-in player discover others and exchange invites cheaply, while a Relay-backed **Party Session** carries the actual networked gameplay. Crucially, every authenticated player **eagerly** hosts their own party session the moment they reach the main menu — the “Always-InParty” model — so an invite is a simple join, never a fragile create-then-hand-off dance.

On top of that sit two more load-bearing rules: **single-writer SOAP** data flow (one service owns each piece of shared state; everyone else reads through ScriptableObject events), and a strict **main-thread affinity contract** (`.AsMainThread()`) at every UGS / Netcode `await` to stop off-thread callbacks from crashing Unity. Player and vessel spawning is **server-authoritative** and flows through one Netcode + SOAP pipeline for menu, AI, and gameplay alike.

2

session layers —
Presence Lobby
(discovery) + Party
Session (Relay
gameplay)

5

UGS services — Auth ·
Sessions · Lobby · Relay
· Friends

7

party lifecycle states in
a single validated state
machine

8

“unbreakable” exit
criteria gating every
commit

The party subsystem was hardened through a **17-commit refactor** that decomposed a 2,000-line orchestrator into one state machine plus nine focused services behind interfaces. The result is a system designed to be **unbreakable** under adversarial conditions — network drops, client crashes, fast input, concurrent invites — with every failure path classified and every catch block mapped to an explicit recovery action.

◆ KEY INSIGHT

Why this is worth reading

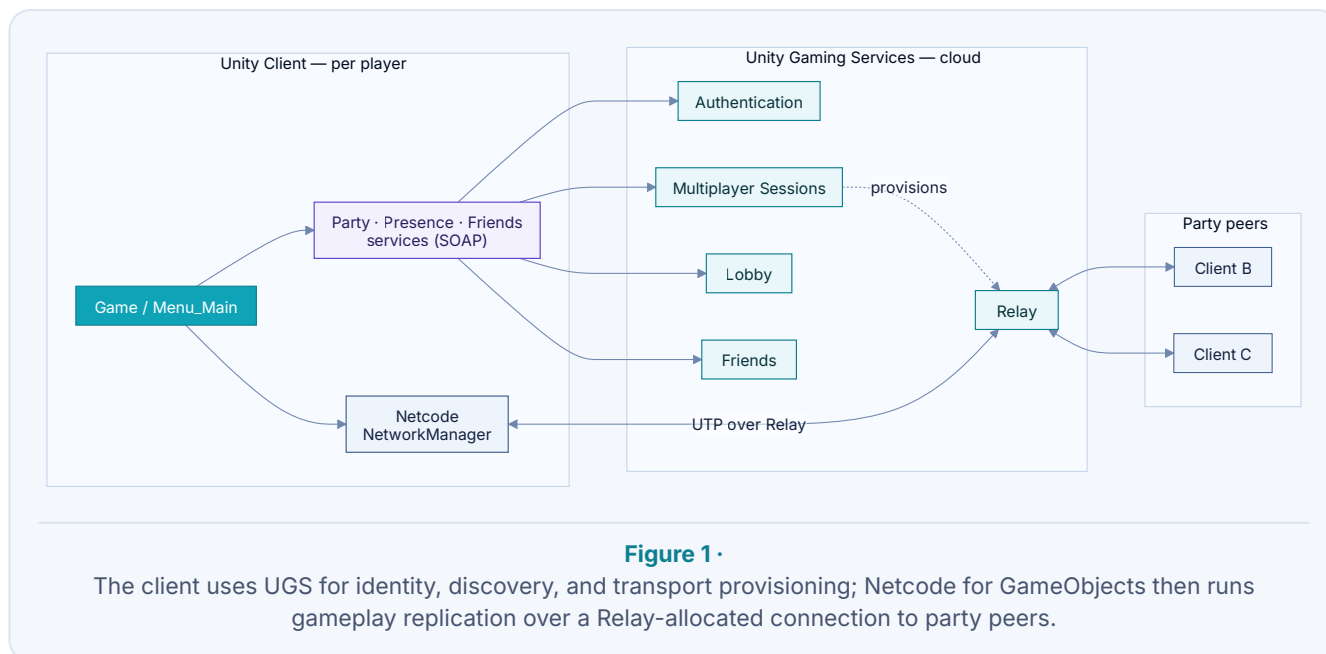
Most of the hard problems here are not Unity-specific — they are the problems of any distributed, event-driven, async system: where state lives, which thread you are on when a remote callback fires, and what happens when two views of “who is in the party” disagree. The fixes are reusable lessons.

How the rest reads. Part I continues with the mental model — the system context, the two-level architecture, the player journey, the key decisions, and a highlight reel of the five most instructive bugs. **Part II** is the engineering reference: each subsystem in full, the complete B-series bug catalogue, the resilience matrix, the threading model, tests, and diagnostics.

PART I · OVERVIEW

System context

Every player runs the same Unity client. Inside it, the game talks to two very different worlds: a set of **Unity Gaming Services** in the cloud, and a peer-to-peer **Netcode** mesh that flows through UGS **Relay** for NAT traversal. The party/presence/friends services are the glue that decides who connects to whom, and Netcode handles what gets replicated once they are connected.



◆ The UGS surface

Service	What we use it for
Authentication	Anonymous sign-in; a stable <code>PlayerId</code> ; cached-session restore. The first thing to complete on boot — everything else waits on it.
Multiplayer Sessions	The unified session API. We create two kinds of session from it: a lobby-only presence session and a Relay-backed party session.
Lobby	Backs the presence session — player lists and per-player properties (how invites travel).
Relay	Allocates a relay server so clients behind NAT can connect without port-forwarding. The party session attaches Relay via <code>WithRelayNetwork()</code> .
Friends	Persistent relationships and rich presence ("In Menu", "In Party", "In Game").

◆ Netcode for GameObjects

Once a party session exists, one player is the **host** (server + client) and the others are **clients**. `NetworkManager` drives connection, scene synchronization, and object replication. The game's `Player` and vessel objects are `NetworkBehaviour`s; their state replicates through `NetworkVariable`s and `ClientRpc` / `ServerRpc` calls. Importantly, the presence lobby is **lobby-only (no Relay)**, so it coexists with a live `NetworkManager` — a player can be discoverable in the lobby and connected for gameplay at the same time without the two interfering.

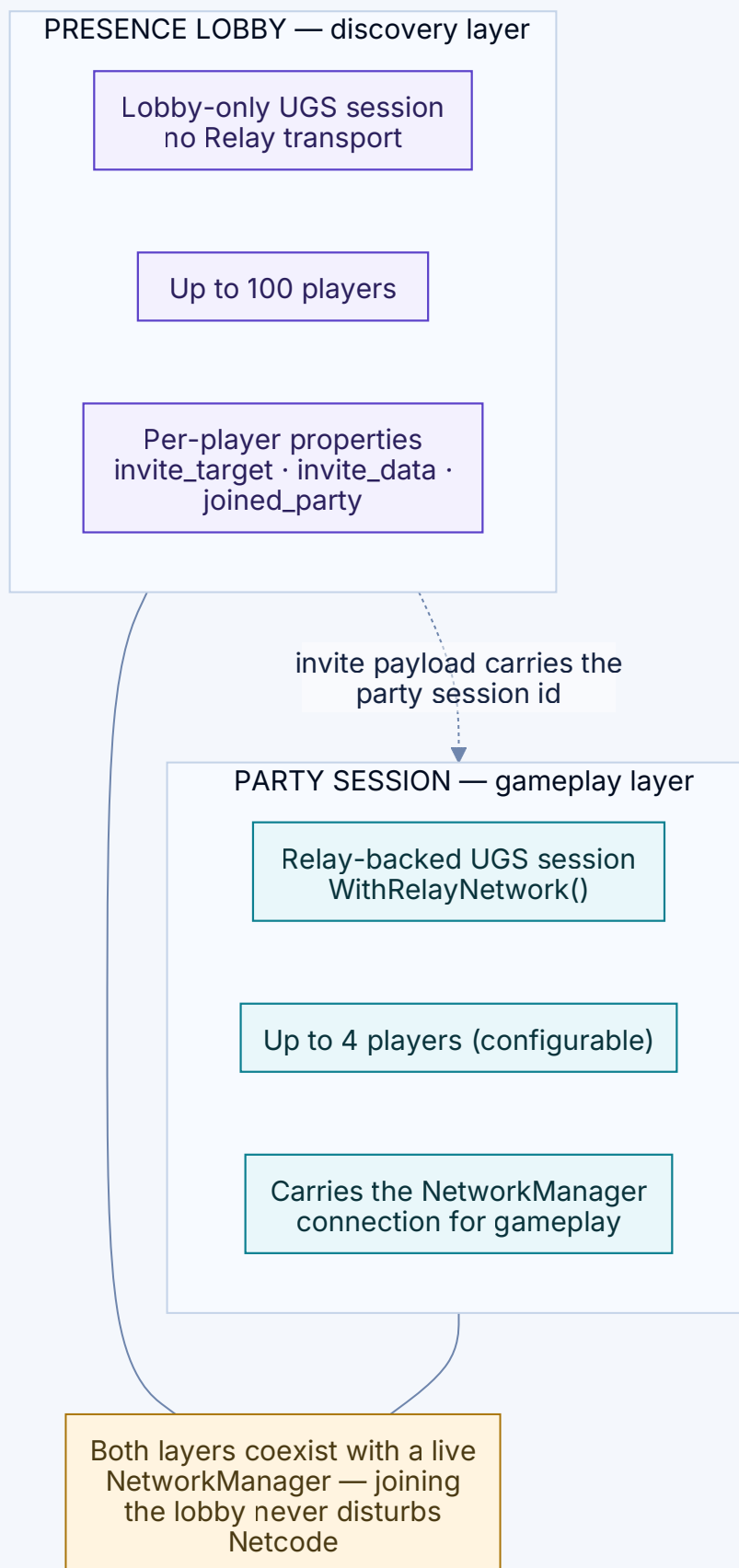
◆ KEY INSIGHT**One client, two clocks**

A lot of the subtlety in this codebase comes from the fact that UGS calls are cloud round-trips whose continuations land on the .NET thread pool, while Netcode and all Unity objects live strictly on the main thread. Bridging those two clocks safely is the subject of the threading model in Part II.

PART I · OVERVIEW

The two-level session model

The single most important architectural choice is to split “being discoverable” from “being connected for gameplay” into two distinct UGS sessions that run side by side.

**Figure 2 ·**

Two UGS sessions per player: a lobby-only Presence Lobby for discovery and invite exchange, and a Relay-backed Party Session for gameplay. The invite payload is the bridge between them.

Layer	Purpose	Relay?	Max players	Service
Presence Lobby	Player discovery, invite property exchange	No (lobby-only)	100	<code>PresenceLobbyService</code>
Party Session	Actual gameplay networking via Relay	Yes (<code>WithRelayNetwork()</code>)	4 (configurable)	<code>PartySessionService</code>

◆ Why separate them

A single Relay-backed session for “everyone online” would be wrong on every axis: Relay allocations are a finite, billed resource, joining one is a slow handshake, and you would not want global discovery traffic flowing over your gameplay transport. Splitting the two means:

- **Discovery is cheap and global.** The presence lobby holds up to 100 players with no Relay cost, and joining it is fast and never disturbs an active `NetworkManager`.
- **Invites need no host privilege.** Invites travel as **per-player properties** on the presence lobby — any member can write their own properties, so any player can invite any other without the lobby host mediating.
- **Gameplay stays small and hot.** The party session is capped at a handful of players and exists only for the people actually flying together.

◆ PITFALL / ANTI-PATTERN

They are not the same thing

A common confusion is to treat “in the lobby” and “in the party” as one state. They are independent: joining the presence lobby does **not** join a party session, and vice-versa. The presence lobby is a *discovery* signal; the party session is the *authoritative* truth of who is connected. When the two disagree, the session always wins — a rule that directly fixed a real host-side bug (B8 in Part II).

◆ How an invite crosses the gap

The presence lobby carries a small set of per-player properties. Two of them are the invite channel, and two more complete the handshake and roster reconciliation:

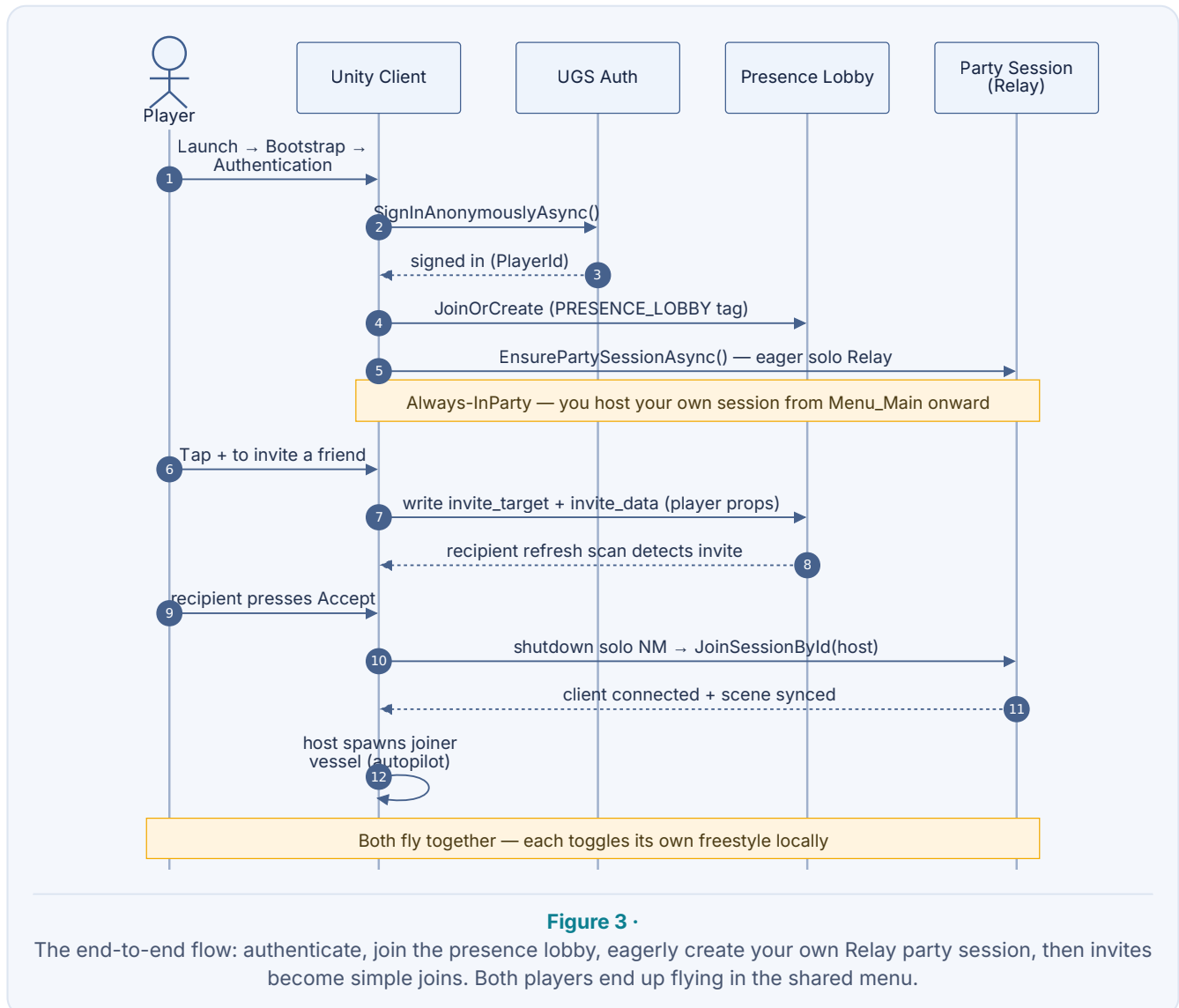
Property	Writer	Meaning
<code>invite_target</code>	Sender	Player ID of the intended recipient
<code>invite_data</code>	Sender	Serialized invite payload (sender’s party session id , display name, avatar)
<code>accepted_invite</code>	Recipient	“I’m coming to join your session” handshake
<code>joined_party</code>	Recipient	“I’m now in this party session” — for host roster reconciliation

Because every player already hosts a party session (next section), the `invite_data` payload always carries a **real** session id. The recipient simply leaves their own session and joins the sender’s.

PART I • OVERVIEW

The player journey

Here is the whole lifecycle, from launch to flying with a friend, as one player experiences it.



- 1. Boot & authenticate.** The Bootstrap scene initialises UGS and signs the player in anonymously, producing a stable `PlayerId`. Nothing networked happens until this completes.
- 2. Become discoverable.** On sign-in, `HostConnectionService` joins the global presence lobby (tagged `PRESENCE_LOBBY`), so the player appears in everyone's "online" list.
- 3. Eagerly host a party.** Immediately, the same service creates the player's **own** Relay-backed party session. The player is now `InParty` — solo, but fully hosted — and a menu vessel spawns on autopilot. This is the "Always-InParty" baseline.
- 4. Send an invite.** Tapping a friend writes `invite_target` + `invite_data` (carrying the real session id) into the sender's presence-lobby properties. No session is created or destroyed.
- 5. Receive an invite.** The recipient's refresh loop (every few seconds) scans lobby properties, finds an invite addressed to it, and raises a SOAP event that pops the Accept / Decline panel.
- 6. Accept = join.** On Accept, the recipient shuts down its own solo session, joins the sender's session over Relay, and waits for the Netcode connection and scene sync to settle.

7. **Spawn & fly.** The host spawns a vessel for the joiner (autopilot on), replicates all existing player-vessel pairs to the new client, and both players fly together in the lava-lamp menu — each independently toggling its own freestyle control.

◆ **KEY INSIGHT**

The invite is a join, not a handoff

Because step 3 already guaranteed a live session with a real id, step 6 is just “leave mine, join yours.” There is no window where a session is being created on demand while another player waits to connect — the class of race conditions that window produced is exactly what the eager model eliminates (see the next section).

Every reversible failure along this path — a join that times out, a dropped connection mid-transition — returns the player cleanly to a solo, controllable menu with no leftover network objects. That reversibility is a first-class requirement, not an afterthought.

PART I · OVERVIEW

Key design decisions

Four decisions carry most of the system's weight. Each is locked — re-opening them is the most common way recurring bugs come back — and each trades something away on purpose.

◆ DESIGN DECISION

EAGER per-user Relay — the “Always-InParty” model

Decision. Every authenticated player creates their own Relay-backed party session the moment they enter the menu, rather than lazily creating one when they send the first invite.

Rationale. Lazy creation meant an invite triggered `create session` → `shut down local host` → `hand off` → `reconnect`, a multi-step cascade that was the root of nearly every recurring party-invite bug. With eager creation an invite is a plain `join` against an id that already exists. **Trade-off.** Every signed-in player holds a Relay allocation even when solo. We accept that cost in exchange for eliminating the shutdown-and-recreate race class entirely.

◆ DESIGN DECISION

Single-writer SOAP for all shared state

Decision. Each piece of cross-system state has exactly one writer. `HostConnectionService` is the sole writer of party/lobby state into `HostConnectionDataSO` ; `FriendsServiceFacade` owns `FriendsDataSO` . Everyone else **reads** through `ScriptableObject` events and lists. **Rationale.** It makes data flow auditable — there is one place to look when “who is in the party” is wrong — and it decouples UI from services entirely (the UI never calls the UGS SDK directly). **Trade-off.** A little ceremony: new state needs a SOAP variable/event and a disciplined single writer, rather than ad-hoc fields. The payoff is no two-writer races and trivially testable UI.

◆ DESIGN DECISION

Main-thread affinity via `.AsMainThread()`

Decision. Every `await` of a UGS or Netcode `Task` is wrapped in `.AsMainThread()` , which re-asserts Unity's `SynchronizationContext` after the call. **Rationale.** UGS continuations resume on the .NET thread pool, where touching any Unity object — or raising a SOAP event whose listeners do — throws `EnsureRunningOnMainThread` . The wrapper guarantees the code after every cloud call is back on the main thread. **Trade-off.** A tiny per-await cost and the discipline to never forget the wrapper — enforced by a runtime canary that logs loudly if a continuation reaches a hot UI path off-thread.

 DESIGN DECISION**Server-authoritative, unified spawning**

Decision. Player and vessel spawning is server-driven and flows through **one** Netcode + SOAP pipeline — the same path for menu autopilot vessels, AI opponents, and live gameplay.

Rationale. One spawn path means one set of bugs to fix. The server owns vessel identity, domain (team), and ownership; clients reactively resolve player-vessel pairs as objects replicate, with zero polling. **Trade-off.** Spawn timing has to be choreographed carefully (deliberate pre/post-spawn delays, and `destroyWithScene` tuning for AI) — but that choreography lives in one place instead of three.

These four compose. Eager sessions keep the state machine simple; single-writer SOAP keeps the UI honest about that state; `.AsMainThread()` keeps the callbacks safe; and server-authoritative spawning keeps who-owns-what unambiguous. The next section shows what happened when each was violated.

PART I · OVERVIEW

Five bugs that taught us the most

The full catalogue is in Part II. These five are the most transferable — each one is a lesson about distributed, async, event-driven systems, not just about Unity.

◆ BUG → ROOT CAUSE → FIX

The off-thread crash — UGS callback raises a SOAP event Fixed

Symptom. Random `EnsureRunningOnMainThread` crashes after party operations, deep in UI code. **Root cause.** A UGS `Task` continuation resumed on the .NET thread pool and raised a SOAP `ScriptableEvent`. SOAP invokes listeners **inline**, so a UI listener touched a `CanvasGroup` off-thread and Unity threw. Worse, the obvious fix — `UniTask.SwitchToMainThread()` — is a no-op on this `UniTask` version (it reports “already complete” from the pool and runs inline). **Fix.** A `MainThreadDispatcher` built on Unity’s own `SynchronizationContext`, exposed as `.AsMainThread()` and applied at every UGS/Netcode await, plus a canary that detects regressions. **Lesson.** Know which thread your callbacks resume on, and don’t trust a library’s thread-switch primitive without verifying it on your version.

◆ BUG → ROOT CAUSE → FIX

LAZY Relay creation — the shutdown-and-recreate cascade Fixed

Symptom. Invites intermittently failed; the second joiner of two often couldn’t connect; vessels sometimes lingered or vanished. **Root cause.** Creating the party session on first invite forced a host shutdown, session create, and reconnect — a cascade with many failure points and timing windows. **Fix.** Make session creation **eager** so every player always has a live session; an invite becomes a join. The entire bug class disappeared. **Lesson.** Sometimes the cheapest fix for a flaky multi-step transition is to delete the transition by making its precondition always true.

◆ BUG → ROOT CAUSE → FIX

The UGS singleton pinned to `null` in a constructor Fixed

Symptom. A service’s calls to `MultiplayerService.Instance` always NRE’d, even long after UGS was initialised. **Root cause.** The service cached `MultiplayerService.Instance` in its **constructor**. Lazy DI singletons are constructed during Bootstrap — before `UnityServices.InitializeAsync()` completes — so the cached value was `null` forever. **Fix.** Resolve at use time via a property: `private IMultiplayerService _svc => MultiplayerService.Instance;` **Lesson.** Never cache a service-locator singleton at construction time when construction can outrun initialisation. Resolve lazily.

◆ BUG → ROOT CAUSE → FIX

Host phantom-rejoin — two views of “who’s in the party” disagree Fixed

Symptom. After a client left, the host’s party slot flickered the departed player in and out forever (~every 3 s), spamming join/leave events. **Root cause.** Two scans disagreed each tick: the authoritative **session** scan correctly removed the player, while a **presence-lobby** scan re-added them from a stale `joined_party` property. **Fix.** Make the presence scan cross-check the authoritative session player list before re-adding — the session is the source of truth; the lobby is only a hint. (Plus an awaited property clear on leave, for hygiene.) **Lesson.** When two data sources can disagree, declare one authoritative and make the other defer to it — never let a hint override truth.

◆ BUG → ROOT CAUSE → FIX

Two vessels and dead controls — spawn vs. scene-reload ordering Fixed

Symptom. After leaving a party, a player’s solo menu showed **two** vessels; the controllable one wouldn’t steer and its AI stopped seeking. **Root cause.** The leave flow recreated the solo session (spawning a vessel) **before** the `Menu_Main` reload finished; the reload’s fresh initializer then spawned a **second** vessel, and the first — a scene survivor (`destroyWithScene=false`) — was orphaned with no player pairing. **Fix.** Despawn the surviving vessel before the reload, and sequence the flow to mirror cold-boot exactly (tear down → leave → shut down NM → load scene → recreate session). **Lesson.** Object lifecycle and scene lifecycle are different clocks; when they overlap during a transition, order them explicitly or you get orphans.

PART I · OVERVIEW

Where it stands

The party system has an explicit definition of “done”: a set of **unbreakable exit criteria** that must hold under adversarial conditions — network drops, client crashes, fast input, and concurrent invites.

#	Criterion	Status
1	No fatal failure — no vessel despawn outside an intentional leave, no kicks, no uncaught UGS exception	Passing
2	No stuck UI — party UI always reflects ground truth	Passing
3	No silent state divergence — host and clients agree within one refresh tick	Passing
4	All transitions reversible — a failed accept returns cleanly to solo menu	Passing
5	Idempotent retries — double-tap Accept can't start two transitions	Passing
6	3-VP accept / decline / leave smoke green every commit	Per-commit gate
7	3-VP stress (5 accepts, random declines/leaves) green	Per-commit gate
8	4-VP concurrent invites — all clients join or bounce cleanly	Per-commit gate

Criteria 1–5 hold as of the 17-commit refactor plus the catch-guard fix. Criteria 6–8 are the active verification gate run on every change via Unity's **Multiplayer Play Mode (MPPM)**, which boots several virtual players in one editor.

◆ What's solid, what's open

- **Solid:** eager session model; single-writer SOAP; the `.AsMainThread()` threading contract; server-authoritative spawning; the validated 7-state party machine; per-catch failure classification; clean leave and clean bounce paths (both MPPM-verified).
- **Open (tracked):** a semaphore-dispose race on fast play-mode abort (B2); a late-second-joiner failure (B5); two presence-lobby discovery edge cases under simultaneous joins (B4, B6). All are documented with reproductions and candidate fixes in Part II.

◆ KEY INSIGHT

Hardening as a discipline, not an event

The system is “hardened toward unbreakable” — an explicit, ongoing posture. Every catch block maps to a named recovery action, every diagnostic line is classified and strippable from release builds, and every locked decision has a written rationale so a future change doesn't silently reintroduce a solved problem. The remaining open bugs are edge cases, not architectural cracks.

The rest of this document is the engineering reference behind all of the above.



PART II

The Comprehensive Deep-Dive

Every subsystem, end to end: the UGS surface, presence lobby, party session and its extracted services, the invite protocol, the Netcode spawn pipeline, threading, SOAP, resilience, the full bug catalogue, testing, and diagnostics.



PART II · FOUNDATIONS

Tech stack & the UGS surface

The online stack rests on two pillars — Unity Netcode for GameObjects for replication, and Unity Gaming Services for identity, discovery, and transport — glued together with UniTask for async and Reflex for dependency injection.

Concern	Technology
Engine / pipeline	Unity 6 + URP 17.0.4
Replication	Unity Netcode for GameObjects 2.5.0
Sessions / Lobby / Relay / Friends / Auth	Unity Gaming Services (Multiplayer SDK)
Async	UniTask (<code>com.cysharp.unitask</code>) — with the project's <code>.AsMainThread()</code> boundary helper
Dependency injection	Reflex 14.1.0 — <code>AppManager</code> is the root installer
Cross-system state	SOAP (Obvious.Soap) — <code>ScriptableVariable</code> / <code>ScriptableEvent</code>
Backend	PlayFab, Firebase, UGS (Analytics, CloudSave, Leaderboards)

◆ The UGS services, in depth

Authentication. Anonymous sign-in via `AuthenticationService`, producing a stable `PlayerId`. A single facade (`AuthenticationServiceFacade`) owns it and writes the result into a SOAP `AuthenticationData` variable. Cached sessions are restored on relaunch. Under Multiplayer Play Mode, the facade switches to a per-instance tagged profile so each virtual player has a distinct identity.

Multiplayer Sessions. The unified `IMultiplayerService` is the factory for both session kinds. A `SessionOptions` without Relay yields the lobby-only presence session; `SessionOptions.WithRelayNetwork()` yields a Relay-backed party session that automatically configures the Netcode transport.

Lobby. Backs the presence session: a roster of players, each with writable per-player properties. Those properties are the entire invite channel — no custom backend required.

Relay. Allocates a relay server and join code so NAT-bound clients can connect. The party session attaches Relay so the host's `NetworkManager` and joining clients communicate over UTP/DTLS without port-forwarding.

Friends. Persistent relationships (`AddFriendByNameAsync`, `AddFriendAsync`), request management, and rich presence — a `FriendPresenceActivity` payload carrying scene, vessel class, and party session id so friends can see exactly what you're doing.

◆ KEY INSIGHT**One SDK, two session shapes**

The cleanest mental model: there is one session API, and the presence of Relay is what makes a session a “party” rather than a “lobby”. Everything else — properties, player lists, refresh — is shared machinery. That symmetry is why the same `IMultiplayerService` live-instance pattern and the same `.AsMainThread()` discipline apply to both layers.

PART II · FOUNDATIONS

Architecture in layers

The party subsystem is deliberately layered so that each concern is testable in isolation and the UI never reaches past the SOAP boundary into a UGS call.

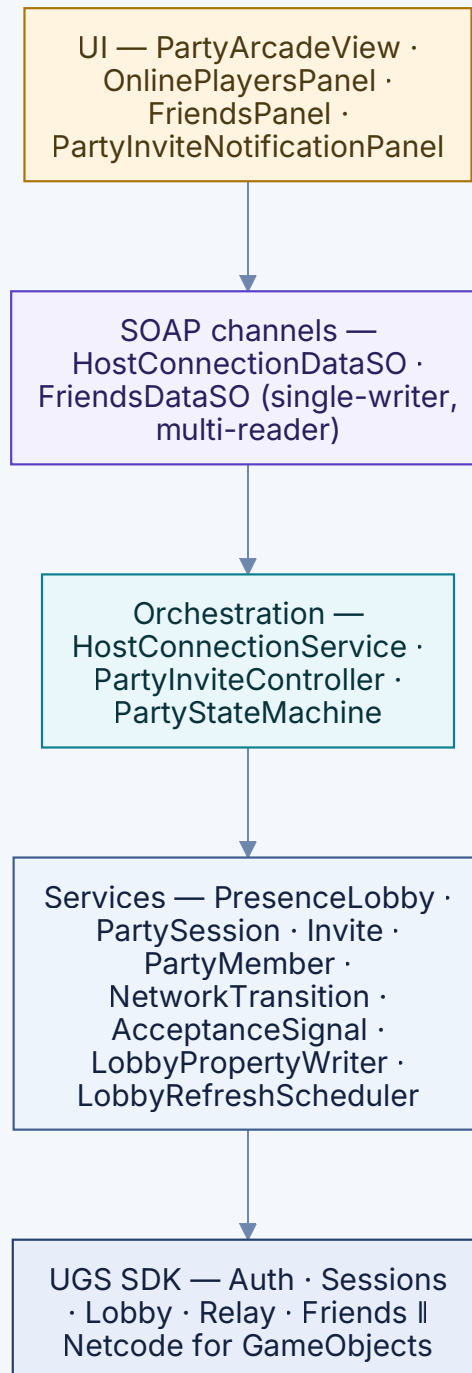


Figure 4 ·

Five layers, top to bottom: UI reads SOAP; SOAP is written by the orchestration layer; orchestration delegates to focused services; services call the UGS SDK and Netcode. Dependencies point downward only.

◆ The orchestration layer

Class	Role
<code>HostConnectionService</code>	The orchestrator and single writer to <code>HostConnectionDataSO</code> . Auto-joins the presence lobby on sign-in, eagerly creates the party session, runs the refresh watchdog, and exposes party operations (<code>AcceptInviteAsync</code> , <code>EnsurePartySessionAsync</code> , <code>KickPartyMemberAsync</code>). <code>DontDestroyOnLoad</code> singleton.
<code>PartyInviteController</code>	The user-facing flow controller. Sequences the Netcode host ↔ client transitions for accept / decline / leave, with linked-CTS timeouts and explicit recovery.
<code>PartyStateMachine</code>	The single source of truth for the party lifecycle phase. Validates every transition against a static table; replaced a drift-prone scatter of boolean flags.

◆ The service layer

The 17-commit refactor extracted nine focused services behind interfaces, so each does one thing:

Service	Responsibility
<code>PresenceLobbyService</code>	Lobby-only session lifecycle — join-or-create, converge-to-canonical, refresh, leave.
<code>PartySessionService</code>	Relay-backed session lifecycle — create / join / leave, with three retry classifiers.
<code>InviteService</code>	Outgoing-invite tracking, payload serialization, timeout.
<code>PartyMemberService</code>	Diffes the SOAP party-member list against the live session player list.
<code>NetworkTransitionService</code>	<code>NetworkManager</code> shutdown and connection/scene-sync waits, all with timeouts.
<code>AcceptanceSignalService</code>	The sender ↔ receiver acceptance handshake.
<code>LobbyPropertyWriter</code>	Mutex-protected, retry-wrapped lobby property writes.
<code>LobbyRefreshScheduler</code>	The polling cadence — base interval plus a post-invite boost window.
<code>SoapPartyEventBus</code>	Centralizes the SOAP event raises so they happen in one auditable place.

◆ KEY INSIGHT

Interfaces are the seams

Every service sits behind an interface (`IPresenceLobbyService` , `IPartySessionService` , `INetworkTransitionService` , `IPartyMemberService` , `IInviteService` , `IPartyStateQuery`). That is what lets the orchestrator be reasoned about — and tested — without spinning up real UGS sessions, and what kept the 2,000-line `HostConnectionService` from growing into something unmaintainable.

PART II · FOUNDATIONS

Authentication & bootstrap

Nothing networked happens until authentication completes. The whole online stack is choreographed off a single SOAP event — `OnSignedIn` — so the auth flow is the spine the rest hangs from.

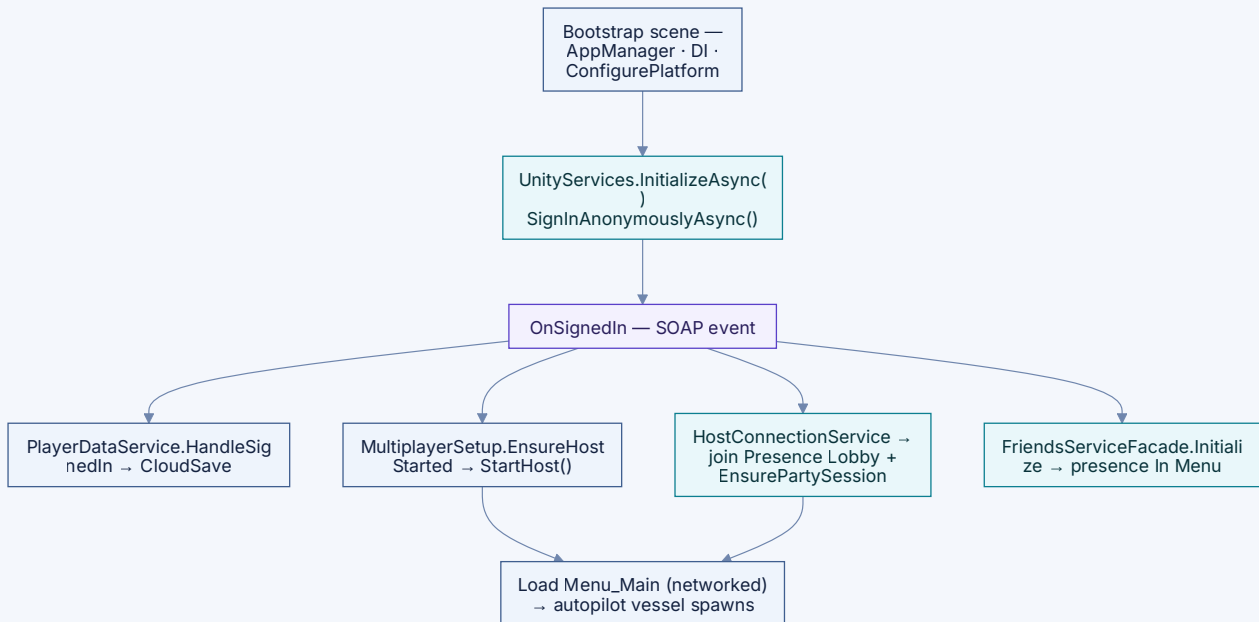


Figure 5 ·

Sign-in fans out: CloudSave profile load, Netcode host start, presence-lobby join + eager party session, and Friends initialisation — all triggered by the one `OnSignedIn` event.

◆ Single-writer authentication

`AuthenticationServiceFacade` is the **sole writer** of `AuthenticationData` (a SOAP variable). It owns UGS initialisation, anonymous sign-in, cached-session restore, event wiring, and sign-out. Every other system — scene controllers, `PlayerDataService`, `MultiplayerSetup`, `HostConnectionService`, `FriendsServiceFacade` — reads auth state or subscribes to its events. None of them mutate it.

CSHARP

```
// The whole stack keys off one event, raised once, on the main thread:
AuthenticationData.OnSignedIn
→ PlayerDataService.HandleSignedIn()           // CloudSave profile load/merge
→ MultiplayerSetup.EnsureHostStartedAsync()    // nm.StartHost() exactly once
→ HostConnectionService.HandleSignedInEvent()  // join presence lobby +
EnsurePartySession
→ FriendsInitializer.HandleSignedInEvent()     // Friends init + presence "In Menu"
```

◆ Bootstrap order

The Bootstrap scene's `AppManager` (`[DefaultExecutionOrder(-100)]` , a Reflex `IInstaller`) registers every persistent service and SO asset, then starts authentication fire-and-forget. Because lazy DI singletons are constructed during this registration — **before** `UnityServices.InitializeAsync()` returns — services must never cache a UGS `*.Instance` at construction time (see the live-instance pattern in the decisions ledger).

◆ PITFALL / ANTI-PATTERN

Idempotency across the boot race

`HostConnectionService` lives in Bootstrap (`DontDestroyOnLoad`) but auth completes later, in the Authentication scene. If auth completes between the service's `Awake` and `Start` , its `Start` calls `HandleSignedInEvent()` directly — which is **idempotent** (it sees the in-progress/initialised state and no-ops). Designing the sign-in handler to be safe to call twice is what makes the boot race a non-event.

The same `OnSignedIn` fan-out runs identically for the host and for a client that later joins a party — there is no special boot path for “joiner”, which keeps the number of distinct startup sequences to exactly one.

PART II · THE DISCOVERY LAYER

The presence lobby

`PresenceLobbyService` owns a UGS **lobby-only** session — no Relay — that every signed-in player joins. It is the discovery layer: up to 100 players, fast to join, and safe to run alongside a live `NetworkManager`.

◆ Join-or-create, and the simultaneous-create race

On sign-in the service queries for an existing lobby tagged `PRESENCE_LOBBY`; if found it joins, else it creates one. Two players signing in at the same instant can both fail to find a lobby and both create one — a split. The service heals this:

- After creating, it **re-queries after a ~1500 ms settle** to detect a rival lobby.
- A periodic **converge-to-canonical** pass (every few seconds) migrates everyone onto a single deterministic “canonical” lobby, collapsing accidental splits back into one.

CSHARP

```
// resolved fresh at use time – never cached in the constructor (the null-pinning pitfall)
private IMultiplayerService _multiplayerService => MultiplayerService.Instance;
```

◆ How invites travel — per-player properties

Invites are encoded as **per-player properties**, not session properties, precisely so no host privilege is needed: any member can write their own properties and thus invite anyone.

Property	Writer	Reader	Meaning
<code>displayName</code> / <code>avatarId</code>	Self	Everyone	UI label and art
<code>partyCount</code> / <code>partyMax</code>	Self	Everyone	“In Lobby N/4” badges
<code>invite_target</code>	Sender	Recipient	Player ID of the recipient
<code>invite_data</code>	Sender	Recipient	Serialized <code>PartyInviteData</code> (sender’s session id + identity)
<code>accepted_invite</code>	Recipient	Sender	“I’m coming to join” handshake
<code>joined_party</code>	Recipient	Everyone	“I’m in this party session” — host roster reconciliation

◆ Refresh cadence

`LobbyRefreshScheduler` drives property reads in two modes: a **base** interval of a few seconds, and a **boost** (~2 s for ~15 s) fired on invite-receive or party-state change to tighten the window for the joiner to see the next state change. The cadence is kept comfortably under the UGS read rate-limit. (The clustering of property writes that boosting can cause is the trigger for the benign SDK stale-index noise documented as B1.)

◆ ForceReset — and why it's dangerous

`ForceReset()` clears the cached lobby reference so the next join-or-create actually runs rather than no-opping. It is called only by the refresh watchdog after the consecutive-error counter trips.

◆ PITFALL / ANTI-PATTERN

A false ForceReset is the classic failure surface

The historical “YS2” bug was an in-flight refresh that fired `ForceReset` during a successful party transition, dumping the joiner into a private throwaway lobby. The fix added a second catch-guard inside the refresh catch block (companion to the entry guard) so a transition in progress suppresses the reset. The lesson: a watchdog that can fire mid-transition must consult “am I transitioning?” before acting.

Everything else reads presence state through SOAP — `HostConnectionDataS0.OnlinePlayers` and the member lists — never by reaching into the service directly.

PART II · THE GAMEPLAY LAYER

The party session & state machine

`PartySessionService` owns the **Relay-backed** session — the one that actually carries gameplay. It is created eagerly per player and is the thing an invite ultimately joins.

◆ One create surface, idempotent

There is exactly one public create-or-no-op entry point: `EnsurePartySessionAsync`. It no-ops if the player is already hosting a party and creates otherwise. The various `RetryCreate*` wrappers that once existed were all deleted — three of their four call sites were really first-time creates, and the fourth (recovery) explicitly clears the stale session first.

`ActiveSession` reads and writes a single backing field on `GameDataSO`, so there is one source of truth for “which session am I in” and it is never nulled outside an intentional leave.

◆ Three retry classifiers

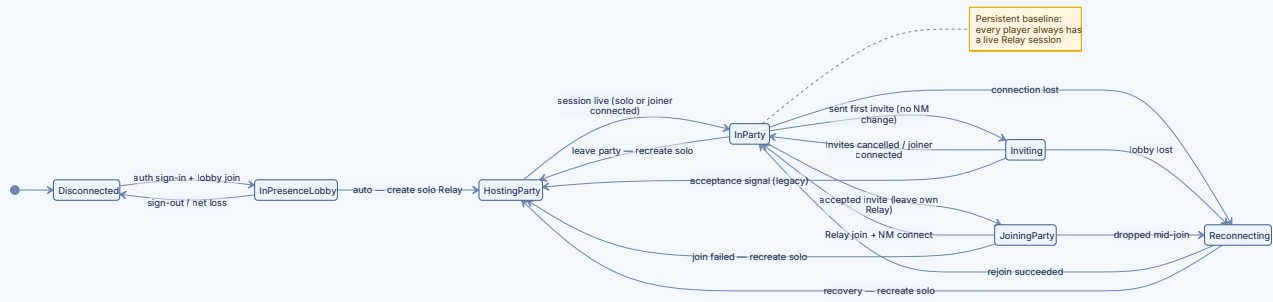
Both create and join run inside retry loops keyed on three exception classifiers — each with its own policy:

Classifier	Retries	Backoff	Covers
<code>IsHostConflictException</code>	up to 2	none	<code>NetworkManager</code> still shutting down from a prior host
<code>IsRateLimitException</code> (HTTP 429)	up to 3	exponential (2 s base)	UGS read/write rate limits
<code>IsTransientSessionException</code>	up to 5	exponential (1 s base)	SDK <code>SessionException</code> NRE, lobby-events 23006, non-fatal collisions

Non-transient errors propagate to `HostConnectionService.AcceptInviteAsync`, which logs and rethrows so `PartyInviteController` fails fast into its recovery path. A freshly-provisioned session can transiently 404 on refresh, so a **4-second post-creation grace period** skips refreshes that would otherwise misclassify a brand-new session as “gone”.

◆ The 7-state lifecycle

`PartyStateMachine` replaced a scatter of booleans (`_initialized`, `_isHost`, `_inviteSent`, `_joining`, `_leaving`) that drifted out of sync across async paths.

**Figure 6 ·**

The validated party lifecycle. `InParty` is the persistent baseline — every player always has a live Relay session. `Disconnected` is reachable from anywhere as the emergency exit.

State	Meaning
<code>Disconnected</code>	Not in any UGS session — initial, sign-out, or fatal error.
<code>InPresenceLobby</code>	Transient — auto-advances to <code>HostingParty</code> as the solo Relay session is created.
<code>HostingParty</code>	Transient — a Relay session is being created/recreated (~1–2 s).
<code>InParty</code>	The persistent baseline — live Relay session (solo or with members), vessel spawned.
<code>Inviting</code>	Sent at least one invite; session already exists, no NM change.
<code>JoiningParty</code>	Accepted someone's invite — shutting down own session, joining the host's.
<code>Reconnecting</code>	Connection lost; recovering back to <code>InParty</code> or down to a fresh <code>HostingParty</code> .

◆ DESIGN DECISION

A state machine over boolean flags

Illegal transitions are rejected with an immediate, explicit warning rather than a silent downstream failure. "Why was the vessel destroyed?" used to trace back to a flag left in the wrong state after a failed transition; now it traces to a single, logged `from → to` timeline visible in the MPPM console.

PART II · THE GAMEPLAY LAYER

The invite protocol, end to end

An invite is a small state exchange over presence-lobby properties, followed by a Netcode host ↔ client transition. Because sessions are eager, the payload always carries a real id.

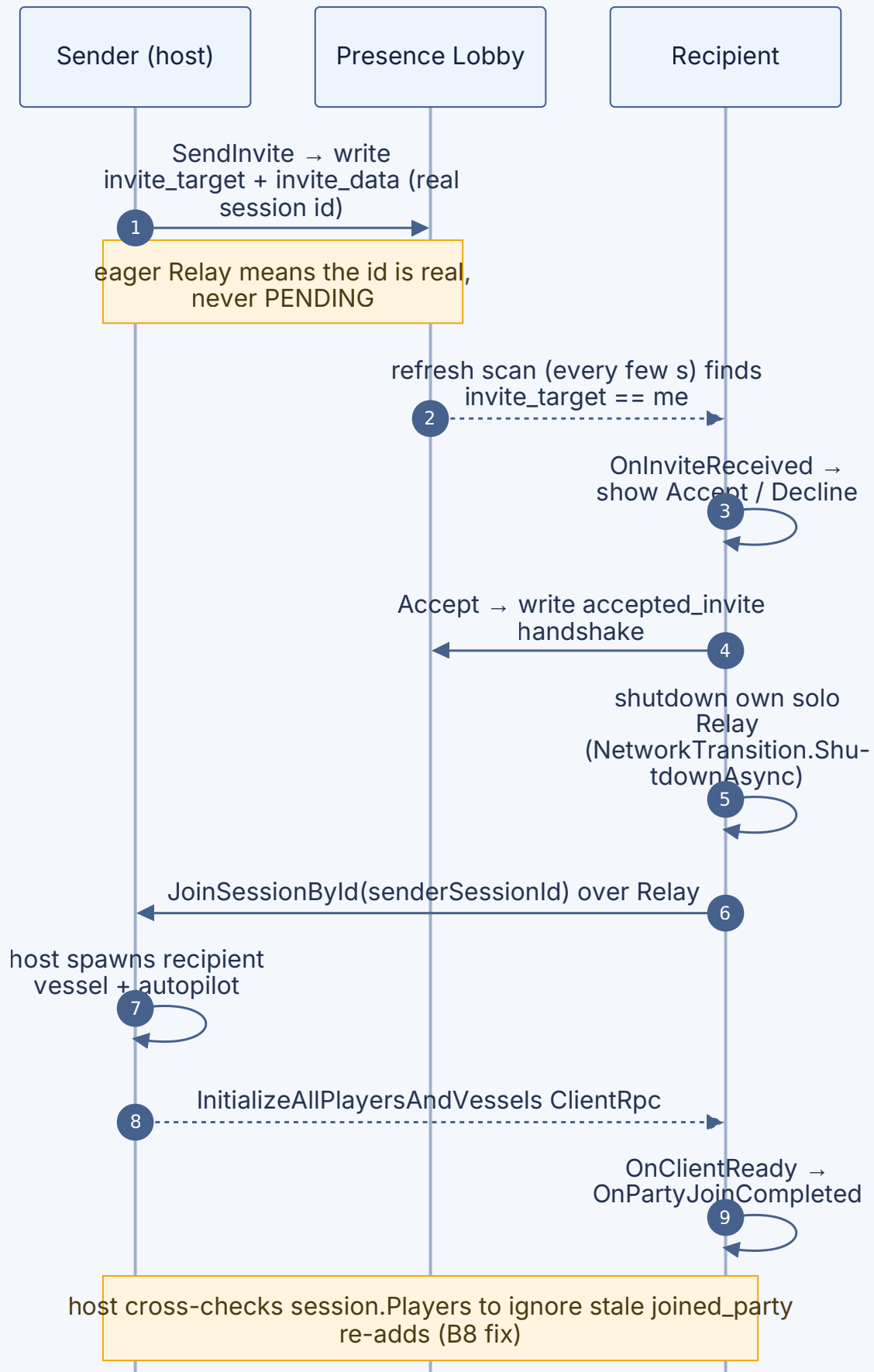


Figure 7 ·

Send → scan → accept → handshake → join. The host spawns the joiner's vessel and replicates all existing pairs; the host's roster scan cross-checks the authoritative session to ignore stale properties.

◆ The payload

`InviteService` serializes each outgoing invite as a single line and writes the set into the sender's lobby properties:

TEXT

```
targetPlayerId | senderPlayerId | sessionId | senderDisplayName | senderAvatarId
```

A legacy `PENDING` sentinel protocol existed for the lazy era — invites could be sent before a session existed, carrying `sessionId = "PENDING"`, and were patched in place once the session came up. With eager creation the id is real from the first write, so the sentinel path is effectively dead — a good example of an architectural decision retiring an entire sub-protocol.

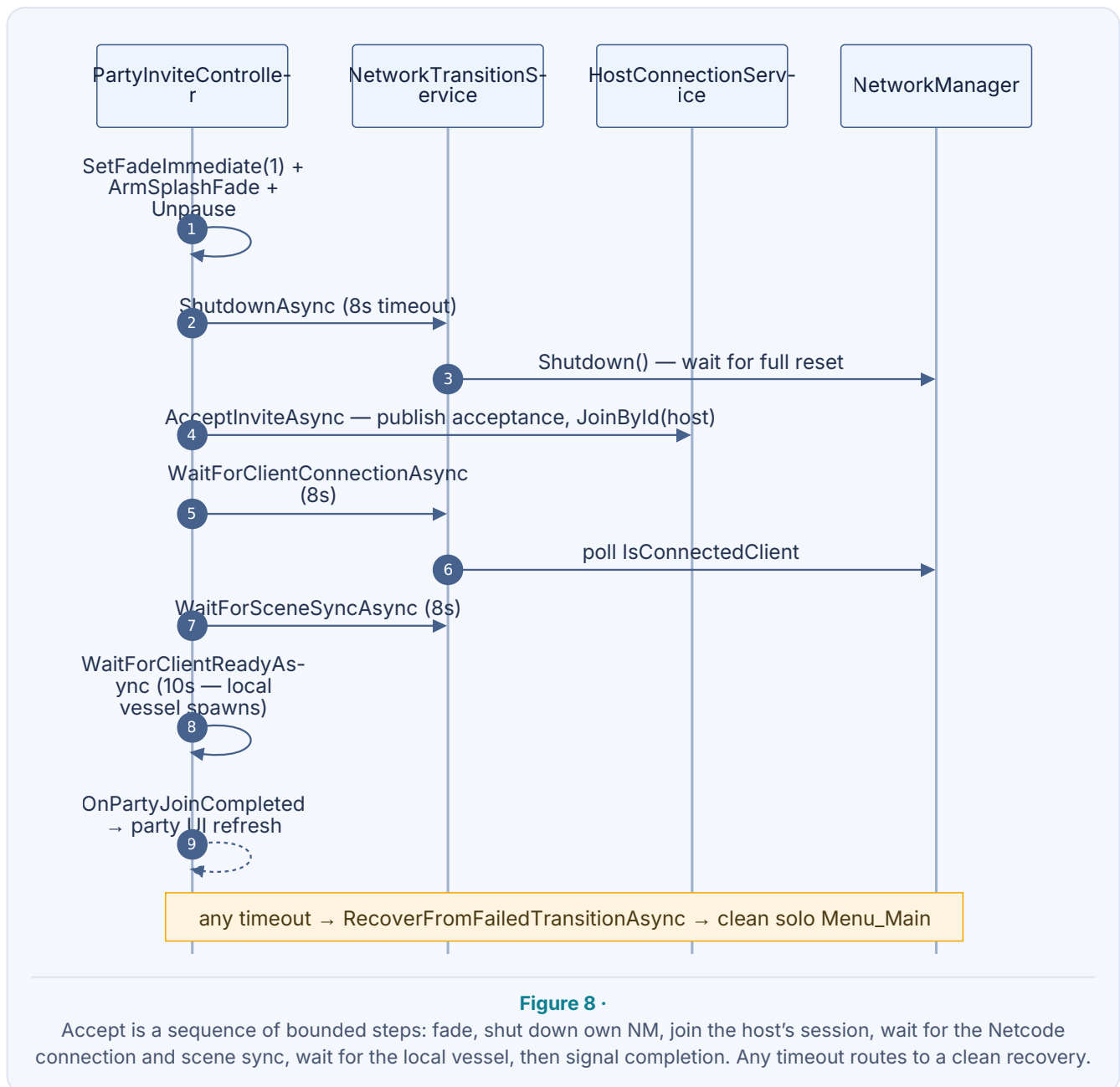
◆ Dedup guards

Properties are read repeatedly by the refresh loop, so the same invite would otherwise fire every tick. Two guards prevent that:

- **SDK-side:** `_lastFiredInvite` / `_lastInviteResolved` caches suppress re-firing `OnInviteReceived` for an invite already surfaced.
- **Lifetime:** an outgoing invite times out after **30 s**, and is cleared on presence-leave or on a successful party-join, so stale invites don't linger.

◆ Accept = a choreographed transition

`PartyInviteController.AcceptInviteAsync` decomposes the host → client switch into explicit primitives, each with its own timeout, rather than one monolithic operation:



1. Transition visuals — fade to black, arm the splash-fade for the next `OnClientReady`, unpause.
2. `NetworkTransitionService.ShutdownAsync` (8 s timeout) — wait for the local host to fully reset.
3. `HostConnectionService.AcceptInviteAsync` — publish the acceptance signal, then `JoinById(host)`.
4. `WaitForClientConnectionAsync` (8 s) — poll `IsConnectedClient`.
5. `WaitForSceneSyncAsync` (8 s) — await the first client `SceneEvent`.
6. `WaitForClientReadyAsync` (10 s) — wait for the local vessel to spawn.
7. Raise `OnPartyJoinCompleted` — party UI refreshes.

◆ KEY INSIGHT**Decompose the transition, make recovery explicit**

Because each step is a named primitive with a timeout, a failure anywhere routes to `RecoverFromFailedTransitionAsync`, which mirrors the clean-leave sequence to return the player to a solo, controllable menu. There is no “stuck mid-transition” state because every step either advances or recovers.

PART II · NETCODE

The player & vessel spawn pipeline

Spawning is server-authoritative and unified: menu autopilot vessels, AI opponents, and gameplay vessels all flow through one pipeline. The server owns identity and ownership; clients reactively resolve pairs as objects replicate.

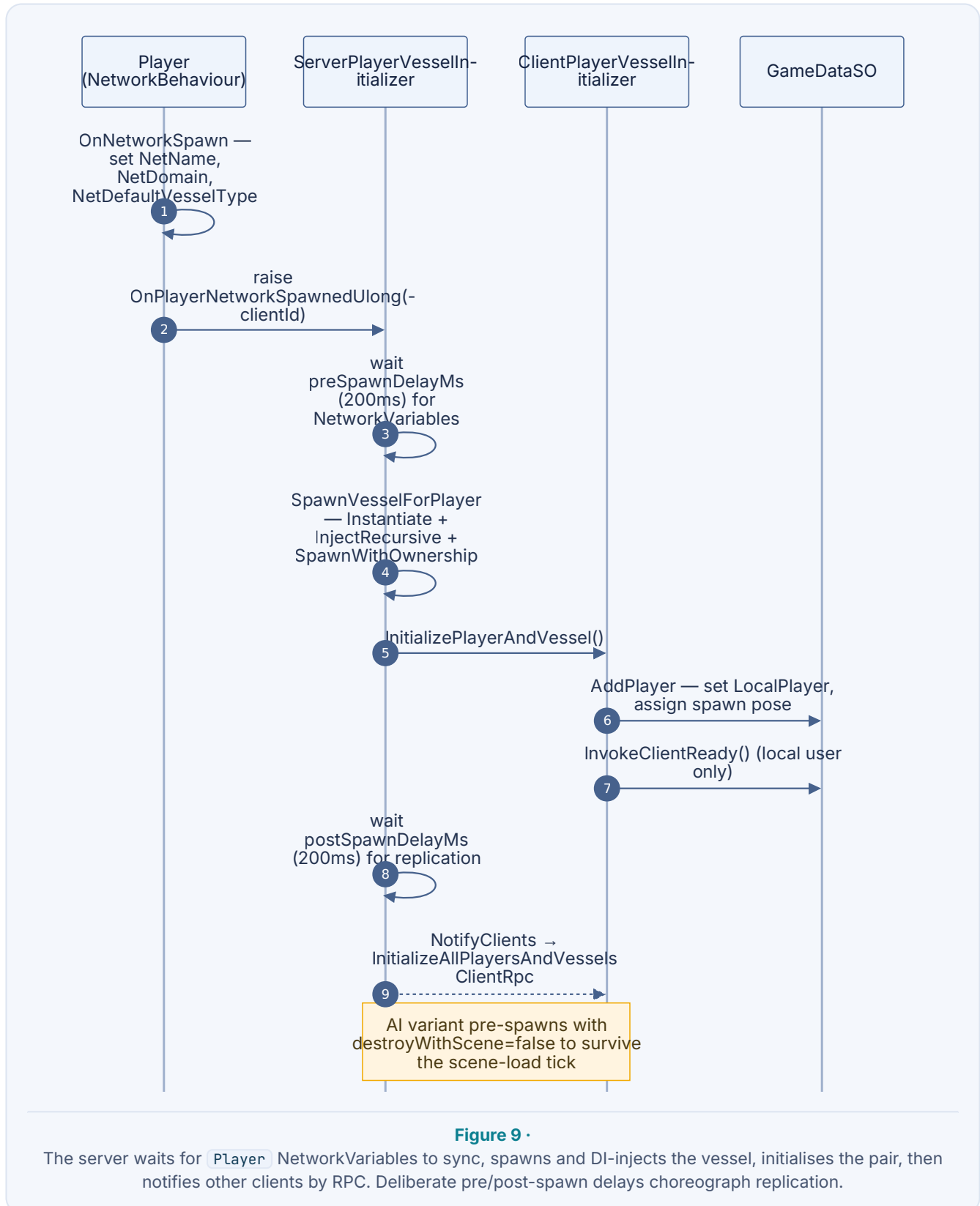


Figure 9 ·

The server waits for **Player** `NetworkVariables` to sync, spawns and DI-injects the vessel, initialises the pair, then notifies other clients by RPC. Deliberate pre/post-spawn delays choreograph replication.

◆ Player is a NetworkBehaviour

Identity replicates through six `NetworkVariable`s, split by writer:

Variable	Writer	Purpose
<code>NetDefaultVesselType</code>	Owner	Vessel class selection
<code>NetName</code>	Owner	Display name (3-tier fallback: profile → cache → UGS)
<code>NetAvatarId</code>	Owner	Profile avatar
<code>NetDomain</code>	Server	Team assignment (via <code>DomainAssigner</code>)
<code>NetVesselId</code>	Server	Linked vessel's <code>NetworkObjectId</code>
<code>NetIsAI</code>	Server	AI flag

Owner-written variables replicate a tick after the object spawns, which is why the server waits `preSpawnDelayMs` (≈200 ms) before reading them, and why a client pair can briefly observe an empty name / `Random` vessel type before identity arrives (documented as the benign B7).

◆ The initializer family

Class	Role
<code>ServerPlayerVesselInitializer</code>	Base server spawner. Listens for <code>OnPlayerNetworkSpawnedUlong</code> , waits for sync, spawns the vessel prefab, DI-injects, then delegates to the client initializer. Tracks processed players by <code>NetworkObjectId</code> .
<code>ServerPlayerVesselInitializerWithAI</code>	Adds AI backfill — pre-spawns server-owned AI before the base subscribes, and balances teams.
<code>MenuServerPlayerVesselInitializer</code>	Adds autopilot activation for menu vessels.
<code>ClientPlayerVesselInitializer</code>	Common pair init; on clients, queues <code>(playerNetId, vesselNetId)</code> pairs from RPCs and resolves them reactively via spawn SOAP events — zero polling.

◆ Two subtleties that caused real bugs

◆ BUG → ROOT CAUSE → FIX

AI vessels destroyed by scene-load tick batching Fixed

AI players spawn with `destroyWithScene: false`. Without it, a client's scene-load message batched on the same network tick as the AI spawn, and the client destroyed the just-spawned AI `NetworkObjects` — surfacing as `[Invalid Destroy]` on the host and invisible AI on clients. Human vessels are unaffected because the 200 ms `preSpawnDelayMs` pushes them to a later tick. Because AI no longer gets free scene-unload cleanup, the controllers explicitly despawn AI before a scene reload.

◆ PITFALL / ANTI-PATTERN**The MPPM connected-client guard**

`LaunchGame`, `ReturnToMainMenu`, and `HandleActiveSessionEnd` all check `if (nm.IsListening && !nm.IsServer) return` after the visual setup but before `LoadSceneAsync`. In Multiplayer Play Mode, SOAP events fire on every virtual player; without this guard a client's `SceneLoader` would race the server's Netcode scene load and destroy AI objects before they replicate. The guard keeps the smooth visuals on clients while deferring the actual load to the server.

PART II · NETCODE

Multiplayer game flow

Once a party launches a game, the same server-authoritative spine drives the match. The base controller synchronises configuration and the turn/round lifecycle; scoring aggregates by **team domain** so AI and human teammates finish together.

◆ Config sync and AI backfill

`MultiplayerMiniGameControllerBase.OnNetworkSpawn` runs `SyncGameConfigToClients_ClientRpc` so every client receives the same intensity, player count, AI-backfill count, and mode. The player-count pipeline is fully data-driven — a party of 2 humans can launch a 12-player game with 10 AI, with no hardcoded caps:

TEXT

```
SO_ArcadeGame (min/max) → stepper UI → GameDataSO.ConfigurePlayerCounts(total, humans)
  → RequestedAIBackfillCount = max(0, total - humans)
  → ServerPlayerVesselInitializerWithAI.SpawnAIs() with team balancing
```

`GetBalancedDomain` assigns each AI to the team with the fewest players, breaking ties by domain enum order (Jade → Ruby → Gold) so identical inputs produce identical AI distributions on every machine — **no shared RNG seed required**.

◆ Domain-aggregated scoring

HexRace, Joust, and Crystal Capture all end on a **per-domain** sum rather than per-player. At most three scores ever exist (Jade / Ruby / Gold); teammates contribute to the same domain total. The turn monitor ends the turn when any active domain's summed total reaches the target, so a human and their AI teammates cross the line together.

◆ KEY INSIGHT

Determinism without a shared seed

Tie-breaking by a fixed enum order instead of RNG is a small decision with an outsized payoff: every client computes the same team assignment from the same inputs, so there is no "AI distribution desync" to debug across machines. It is the same instinct as making sessions eager — remove the nondeterminism rather than synchronise it.

Cross-client domain identity is itself driven by one server-written `NetworkVariable` (`Player.NetDomain`), whose replication callback fans the change out to the local mirror, the server-side round stats, and the vessel materials — so scoreboards, end-game UIs, and team colours stay live across re-picks and AI fills. (Full detail is in the project's domain-sync notes; it is called out here because it is the same single-writer-then-fan-out pattern the party system uses.)

PART II · SOCIAL

The friends system

Friends use the UGS Friends SDK and follow the same single-writer / multi-reader SOAP pattern as auth and party. `FriendsServiceFacade` is the sole writer of `FriendsDataSO`; UI reads four reactive lists and four events.

◆ Facade API

Every mutating method calls `SyncAllRelationships()` after the SDK call to keep the SOAP lists honest.

Method	UGS SDK call	Effect
<code>InitializeAsync</code>	<code>FriendsService.InitializeAsync</code>	Wire events, sync lists, raise <code>OnFriendsServiceReady</code>
<code>SendFriendRequestByNameAsync</code>	<code>AddFriendByNameAsync</code>	Add to <code>OutgoingRequests</code>
<code>SendFriendRequestAsync</code>	<code>AddFriendAsync</code>	Add to <code>OutgoingRequests</code>
<code>AcceptFriendRequestAsync</code>	<code>AddFriendAsync</code>	Move incoming → <code>Friends</code> , raise <code>OnFriendAdded</code>
<code>DeclineFriendRequestAsync</code>	<code>DeleteIncomingFriendRequestAsync</code>	Remove from <code>IncomingRequests</code>
<code>RemoveFriendAsync</code>	<code>DeleteFriendAsync</code>	Remove from <code>Friends</code> , raise <code>OnFriendRemoved</code>
<code>BlockPlayerAsync</code> / <code>UnblockPlayerAsync</code>	<code>AddBlockAsync</code> / <code>DeleteBlockAsync</code>	Manage <code>BlockedPlayers</code>
<code>SetPresenceAsync</code>	<code>SetPresenceAsync</code>	Update rich presence
<code>RefreshAsync</code>	<code>ForceRelationshipsRefreshAsync</code>	Full server refresh

◆ Rich presence across scene transitions

`FriendsInitializer` is the only thing that sets presence, so it stays consistent:

Trigger	Availability	Activity
Enter menu / sign-in	<code>Online</code>	"In Menu" (<code>Menu_Main</code>)
Enter a game scene	<code>Busy</code>	"In Game" (scene, vessel class, party session id)
App shutdown	<code>Offline</code>	—

◆ Two ways to add a friend

These are intentionally different because the caller has different information:

Entry point	Input	Facade method	Where
<code>AddFriendPanel</code>	Player name (text)	<code>SendFriendRequestByNameAsync</code> <code>c</code>	Friends panel → Add tab
<code>OnlinePlayerEntry</code>	Player ID (from presence)	<code>SendFriendRequestAsync</code>	Online players panel → per-row "+"

◆ KEY INSIGHT

Friends vs. party invites are different systems

Adding a friend is a **persistent** relationship via the Friends SDK; a party invite is an **ephemeral**, session-scoped exchange via lobby player properties. They can appear on the same UI row but they never share state — and the friend system feeds the party system (a friend row's "invite" button calls `HostConnectionService.SendInviteAsync`), rather than duplicating it.

PART II · CROSS-CUTTING

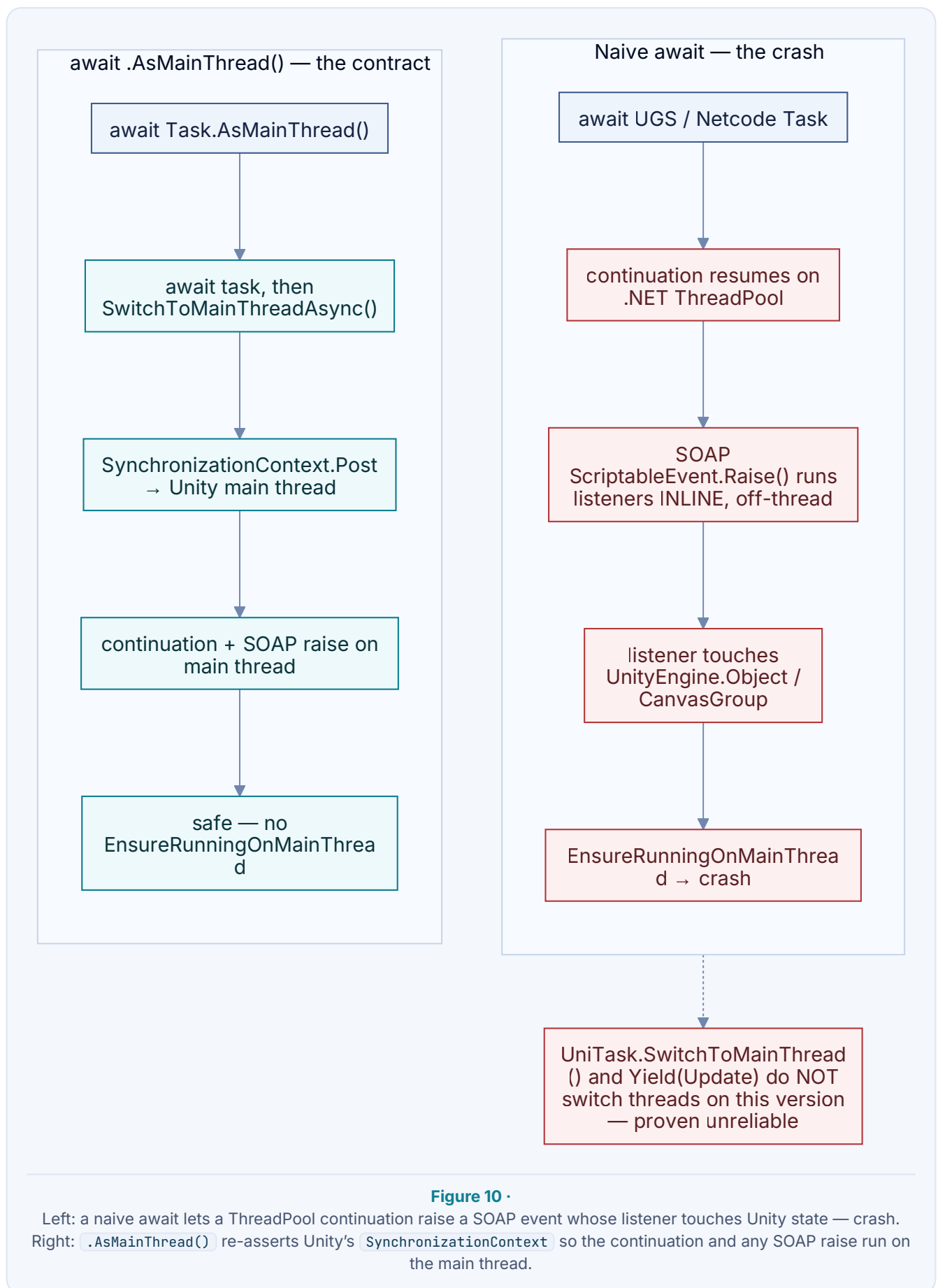
Threading & main-thread affinity

This is the subtlest part of the stack and the source of the hardest-won lesson. The one-sentence rule: **wrap every cross-thread `await` in `.AsMainThread()`** — and do not reach for `UniTask`'s own thread-switch primitives, because they don't work reliably on this version.

◆ The problem

UGS and Netcode methods return `System.Threading.Tasks.Task`. Their continuations land on whatever thread the SDK's HTTP/WebSocket pump completes on — typically the .NET ThreadPools. From there:

- Any `UnityEngine.Object` access — including `obj = null`, which routes through `Object.op_Equality → EnsureRunningOnMainThread` — throws.
- Any SOAP `ScriptableEvent.Raise()` invokes its listeners **inline on the calling thread**, so a UI listener that touches a `CanvasGroup` crashes one level deeper.



◆ Why the obvious `UniTask` primitives don't fix it

`UniTask 2.x` intentionally bypasses `SynchronizationContext` ("UniTask always works like `Task.ConfigureAwait(false)`"). The consequence, verified on this version:

Primitive	What it does here	Verdict
<code>UniTask.SwitchToMainThread()</code>	Awaiter reports complete from the pool → continuation runs inline on the pool. No switch.	Broken
<code>UniTask.Yield(PlayerLoopTiming.Update)</code>	Yields, but the continuation queue doesn't marshal through the <code>SyncContext</code> , so it can resume on a worker.	Broken
<code>UniTask.NextFrame()</code> / <code>DelayFrame(1)</code>	Same bypass internally.	Broken

◆ The fix

A `MainThreadDispatcher` built on **Unity's own** `SynchronizationContext`, captured before any scene loads:

CSHARP

```
public static class MainThreadDispatcher
{
    static SynchronizationContext _mainContext;
    static int _mainThreadId;

    [RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.BeforeSceneLoad)]
    static void Init()
    {
        _mainContext = SynchronizationContext.Current; // Unity's context
        _mainThreadId = Thread.CurrentThread.ManagedThreadId;
    }

    public static bool IsOnMainThread ⇒
        Thread.CurrentThread.ManagedThreadId == _mainThreadId;

    public static UniTask SwitchToMainThreadAsync()
    {
        if (IsOnMainThread) return UniTask.CompletedTask;
        var tcs = new UniTaskCompletionSource();
        _mainContext.Post(_ ⇒ tcs.TrySetResult(), null); // drains on main thread
        return tcs.Task;
    }
}
```

`.AsMainThread()` is the boundary helper callers actually use — four overloads cover `Task`, `Task<T>`, `UniTask`, and `UniTask<T>`, so the surface is identical regardless of what the SDK returns:

CSHARP

```
// Continuation guaranteed on the main thread:  
ActiveSession = await MultiplayerService.Instance.CreateSessionAsync(opts).AsMainThread();  
connectionData.IsPartyHost = true; // safe - raising SOAP here won't crash
```

◆ The canary

`SceneManager.SetFadeImmediate` is on a hot path and touches a `CanvasGroup`. It checks `MainThreadDispatcher.IsOnMainThread` and logs a loud `Debug.LogError` (naming the helper to add) if it is ever reached off-thread. If a future feature forgets a `.AsMainThread()`, the canary fires immediately instead of producing a mysterious crash later.

◆ KEY INSIGHT

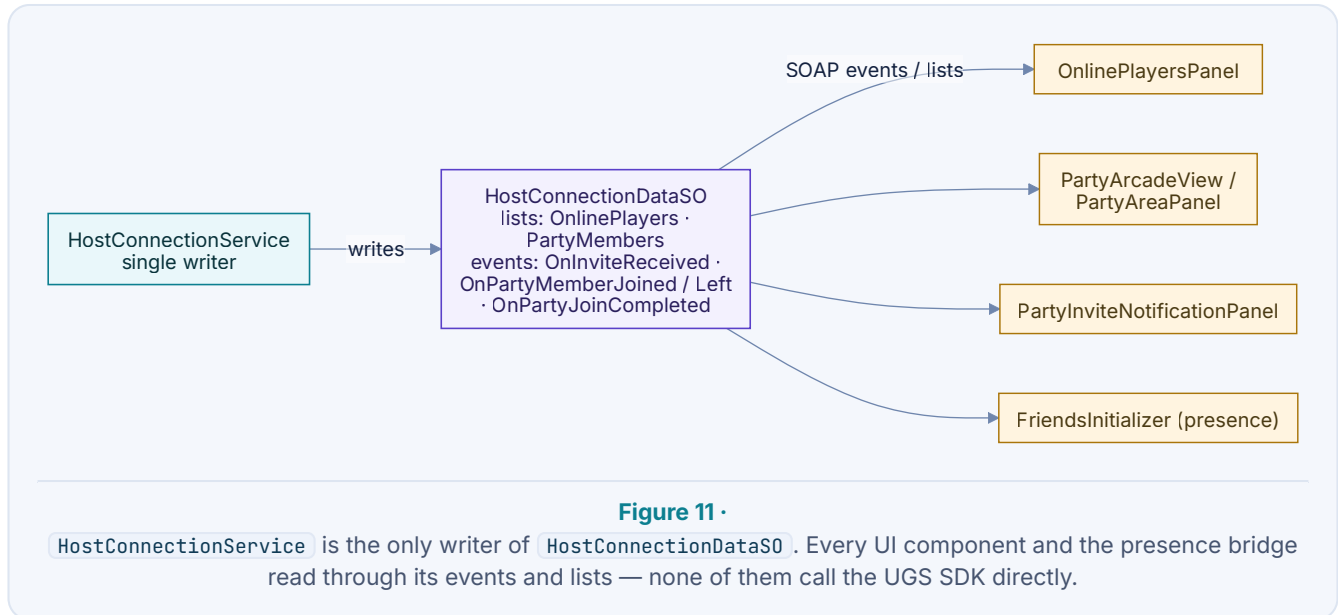
The history is the warning

The fix took seven commits — two broad `SwitchToMainThread` migrations and a `YieldUpdate` attempt were all tried and reverted before the `SynchronizationContext` approach stuck. That history is kept in the docs precisely so the next person who reaches for `SwitchToMainThread()` stops and reads first.

PART II · CROSS-CUTTING

SOAP data architecture

SOAP (Scriptable Object Architecture Pattern) is how the whole project does cross-system communication: shared state lives in `ScriptableObject` assets, notifications travel through `ScriptableEvent` channels, and UI reacts via inspector-wired listeners. The party system uses it with one strict rule — **single writer, many readers**.



◆ `HostConnectionDataSO` — the party data container

Kind	Members
Lists	<code>OnlinePlayers</code> , <code>PartyMembers</code> (self at index 0)
Events	<code>OnHostConnectionEstablished</code> , <code>OnHostConnectionLost</code> , <code>OnPartyMemberJoined</code> , <code>OnPartyMemberLeft</code> , <code>OnPartyMemberKicked</code> , <code>OnInviteReceived</code> , <code>OnInviteSent</code> , <code>OnPartyJoinCompleted</code> , <code>OnInviteResolved</code>
Runtime state	<code>LocalPlayerId</code> , <code>LocalDisplayName</code> , <code>LocalAvatarId</code> , <code>IsConnected</code> , <code>IsPresenceLobbyHost</code> , <code>IsPartyHost</code>

◆ `FriendsDataSO` — the friends container

Four reactive lists (`Friends`, `IncomingRequests`, `OutgoingRequests`, `BlockedPlayers`) and four events (`OnFriendAdded`, `OnFriendRemoved`, `OnFriendRequestReceived`, `OnFriendsServiceReady`).

◆ The SOAP value types

Type	Shape
<code>PartyInvite</code> Data	<code>HostPlayerId</code> , <code>PartySessionId</code> , <code>HostDisplayName</code> , <code>HostAvatarId</code>
<code>PartyPlayer</code> Data	<code>PlayerId</code> , <code>DisplayName</code> , <code>AvatarId</code> , <code>PartyMemberCount</code> , <code>PartyMaxSlots</code> , <code>MatchName</code> — equality by <code>PlayerId</code> only, so party-state fields can update without breaking dedup

Type	Shape
FriendData	PlayerId, DisplayName, Availability, ActivityStatus

◆ PITFALL / ANTI-PATTERN

Fail loud — no null-guards on SOAP event fields

Project policy forbids `if (evt ≠ null)` guards on serialized `ScriptableEvent` fields. A missing reference is a wiring error that should crash immediately and obviously, not silently no-op and hide the bug. The same philosophy runs through the whole stack: loud, traceable failures over quiet divergence.

◆ KEY INSIGHT

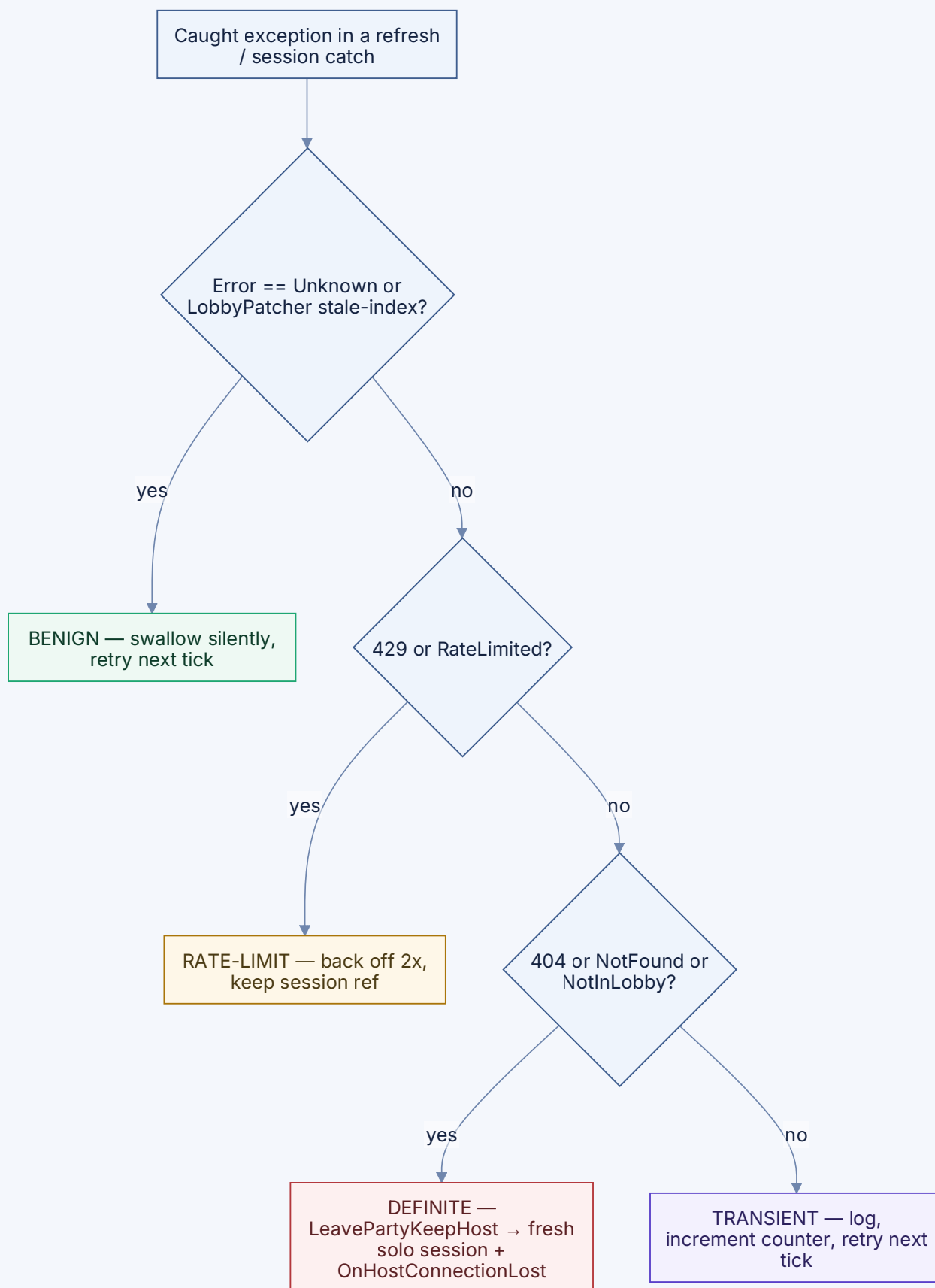
Why single-writer SOAP is load-bearing here

With one writer, “who is in the party is wrong” has exactly one place to investigate. UI is fully decoupled — it can be tested against a fabricated `HostConnectionDataS0` with no UGS at all — and there are no two-writer races on shared lists. It is the data-flow analogue of the state machine: one owner, validated changes, observable everywhere.

PART II · CROSS-CUTTING

Resilience & the error matrix

The unbreakable invariant — the UI must never show “in party” when there is no party — is enforced by classifying every failure and mapping it to an explicit recovery. **No catch silently drops state; no catch leaves the system worse than it entered.**

**Figure 12 ·**

Every refresh/session exception is classified into one of four buckets, each with a defined recovery. The presence-lobby SDK's stale-index noise is recognised as benign and swallowed.

◆ Recovery actions by failure class

Failure class	Detection	Recovery
Benign	<code>SessionException.Error = Unknown</code> ; <code>LobbyPatcher</code> stale-index	Swallow silently — known self-correcting SDK churn
Rate-limit	"Too Many Requests" / HTTP 429	Back off (2× interval), keep the session ref, retry next tick
Definite (session gone)	404 / <code>SessionNotFound</code> / <code>SessionDeleted</code> / <code>NotInLobby</code>	<code>LeavePartyKeepHost</code> → fresh solo session; raise <code>OnHostConnectionLost</code> + per-member <code>OnPartyMemberLeft</code> so UI updates
Transient	everything else	Log, increment the consecutive-error counter, retry next tick; after a threshold, promote to definite

◆ The timing constants that make it robust

Constant	Value	Why
Base refresh interval	a few seconds	Under the UGS read rate-limit
Boost window	~2 s for ~15 s	Tightens propagation right after an invite
Post-creation grace	4 s	A fresh session can transiently 404 — don't misclassify it as gone
Outgoing invite timeout	30 s	Stale invites expire automatically
Host-conflict retries	2	NM still shutting down
Rate-limit retries	3, exp backoff	Cloud throttling
Transient retries	5, exp backoff	SDK NRE / lobby-event collisions
Clear-joined-party timeout	3 s	Bounded — a clean leave must never hang on a flaky property write

◆ KEY INSIGHT

Classify, don't crash

The key discipline is that recovery policy ("what to do") is decided by a small set of exception classifiers, and is kept deliberately separate from the diagnostic classifier ("what to log", next section). They can diverge legitimately — a failure can be interesting to log but not worth a retry — and keeping them apart stops log format from accidentally driving retry behaviour.

The benign bucket deserves a note: the same UGS stale-index defect surfaces three different message strings across restarts, so the code keys off the **structured** `Error = Unknown` reason rather than message text — message-matching turned into whack-a-mole, while every actionable failure carries a specific reason that the definite/rate-limit branches catch first.

PART II · THE RECORD

The bug catalogue

The party and presence layers keep living bug trackers, found through Multiplayer Play Mode (MPPM) testing. Statuses follow the project legend.

ID	Layer	Title	Status
B1	Presence	<code>LobbyPatcher</code> stale-index <code>ArgumentOutOfRangeException</code> spam at game start	Mitigated
B2	Party	<code>ObjectDisposedException</code> (semaphore) on play-mode abort / fast accept	Open
B3	Party	Bounce leaves two vessels + dead controls	Open
B3.b	Party	Same symptom on the clean-leave path	Fixed
B4	Presence	Second invite not delivered + members vanish from 3rd player	Open
B5	Party	Second joiner fails to join	Open
B6	Presence	<code>WrappedLobbyService</code> NRE + empty online/request lists	Mitigated
B7	Party	Client pair-init runs before remote identity replicates	Deferred (benign)
B8	Party	Host phantom-rejoin loop from stale <code>joined_party</code>	Fixed

◆ The instructive fixes

◆ BUG → ROOT CAUSE → FIX

B8 — host phantom-rejoin from divergent truth Fixed

After a client left, the host re-added them every refresh tick (~3 s) from a stale `joined_party` presence property, even as the authoritative session scan correctly removed them — a join/leave flicker forever. **Fix 1 (load-bearing)**: the presence scan now cross-checks the live session player set and skips anyone not in it — the session is truth, the lobby is a hint. **Fix 2 (hygiene)**: the leave path now awaits the property clear (bounded to 3 s) so the wire is correct too. Shipped as separate commits so a regression can be bisected.

◆ BUG → ROOT CAUSE → FIX

B3.b — two vessels + dead controls on leave Fixed

Leaving a party recreated the solo session (spawning a vessel) **before** the `Menu_Main` reload finished; the reload's fresh initializer spawned a second vessel, and the first survived as an orphan (`destroyWithScene=false`) with no player pairing — uncontrollable, AI idle. An earlier band-aid was **rejected** in favour of the real fix: despawn the surviving vessel before the reload and sequence the flow to mirror cold-boot exactly. Root cause was confirmed from a sequential MPPM trace showing the two spawns bracketing the scene-reload boundary.

◆ BUG → ROOT CAUSE → FIX

B1 — benign SDK stale-index noise Investigating

The UGS Lobby SDK logs an `ArgumentOutOfRangeException` from `LobbyPatcher` when a WebSocket delta references a stale local index — thrown and logged by the SDK before any of our `await` s, so a try/catch can't suppress it. A `BenignLobbyLogFilter` (editor/dev only) drops exactly that signature; the same defect on the save/get API surfaces is silenced at our own catches by matching the **structured** `Error = Unknown` reason (message-matching was abandoned after three message variants of the same defect appeared across three restarts). It is self-correcting — pure console hygiene.

◆ The open ones (with reproductions)

- **B2** — `RefreshAsync` holds `_lobbyMutex` while an awaited UGS save completes frames later; `OnDestroy` disposes the semaphore in between, so the late `finally` releases a disposed semaphore. Candidate fix: a `_destroyed` guard + cancellation, or simply not disposing the semaphore.
- **B5** — the second of two joiners times out at `WaitForClientReadyAsync` (the host's server-side vessel-spawn / roster path never reaches `OnClientReady`), then bounces. Diagnose-first via NetDiag.
- **B4 / B6** — presence-lobby discovery edge cases under simultaneous joins (convergence pause freezing a lobby split; a stale-ref NRE emptying lists). These are the fragile, locked-design area — read the presence architecture before touching it.

◆ KEY INSIGHT

How bugs are worked

One at a time, in priority order, each on its own commit with a risk table and a status update. Root cause is confirmed via NetDiag log capture *before* a fix is written — the project's strong bias is to understand the mechanism, not to suppress the symptom. The rejected B3.b band-aid is the canonical example of that bias in action.

PART II · VERIFICATION

Testing strategy

The party system is verified at three levels: fast edit-mode unit tests for the data and control logic, a play-mode test for the accept flow, and manual Multiplayer Play Mode (MPPM) procedures for the multi-client behaviour that can't be unit-tested.

◆ Automated tests

Suite	Scope
<code>PartyInviteDataTests</code>	Invite payload value semantics
<code>PartyPlayerDataTests</code>	Player-data equality (by <code>PlayerId</code>) and party-state fields
<code>PartyInviteControllerTests</code>	Controller transition logic (uses reflection on <code>_transitioning</code>)
<code>PartyInviteSystemTests</code>	Invite system integration
<code>HostConnectionDataS0Tests</code>	SOAP container behaviour
<code>DomainAssignerTests</code>	Team assignment / balancing determinism
<code>PartyAcceptFlowPlayModeTests</code>	The accept flow under a real play-mode network

◆ KEY INSIGHT

Test the seams, not the cloud

The interface-per-service decomposition is what makes the orchestrator testable without standing up real UGS sessions. The control logic — state transitions, payload parsing, member diffing, team balancing — is exercised in fast edit-mode tests; only the genuinely networked accept flow needs a play-mode harness.

◆ Manual MPPM procedures

Behaviour that depends on several real clients is covered by documented manual procedures, run in Unity's Multiplayer Play Mode (several virtual players in one editor):

- **Party smoke / stress (S-series):** accept, decline, leave, and a stress run of five consecutive accepts with random declines/leaves interleaved.
- **Presence procedures (P-series):** discovery, simultaneous-create convergence, invite delivery across a lobby split.
- **Diagnostics procedures (A-E):** verify the NetDiag classifier and `NetworkMonitor` accuracy, including the case where the Editor's `internetReachability` lies.

A chronological **MPPM session log** captures session-scoped findings — the timeline view matters because many bugs (like B3.b and B8) were only understood by reading the exact ordering of log lines across two clients.

◆ The per-commit gate

The eight unbreakable exit criteria are the acceptance bar. Criteria 1–5 (no fatal failure, no stuck UI, no silent divergence, reversible transitions, idempotent retries) are demonstrated to hold; criteria 6–8 (3-VP smoke, 3-VP stress, 4-VP concurrent invites) are re-verified per commit. A change isn't "done" until that gate is green.

PART II · VERIFICATION

Network diagnostics

Every party / lobby / session / transition catch block appends a one-line **NetDiag** record next to its existing log. It is pure observability — adding a call site never changes behaviour — and it is what turned “the invite sometimes fails” into “the invite failed because the host’s server-side spawn never completed”.

◆ The record

TEXT

```
NetDiag: class=<category> | reach=<UnityReachability> | monitor=<state> |
sinceChange=<seconds>
```

Field	Meaning
<code>class</code>	Exception category from <code>NetworkDiagnostics.ClassifyException</code> (or a hard-coded <code>Timeout</code> at the three transition timeouts)
<code>reach</code>	<code>Application.internetReachability</code>
<code>monitor</code>	Live <code>NetworkMonitor</code> state (<code>Online</code> / <code>Offline</code> / <code>Uninitialized</code>)
<code>sinceChange</code>	Seconds since the last monitor transition

◆ Classification rules (first match wins)

Input	<code>class=</code>
<code>OperationCanceledException</code> / <code>TaskCanceledException</code>	<code>Cancelled</code>
<code>AuthenticationException</code>	<code>AuthRequired</code>
<code>SessionException</code> containing <code>NotFound</code> / <code>NotInLobby</code>	<code>SessionGone</code>
<code>SessionException</code> otherwise	<code>Transient</code>
<code>LobbyServiceException</code> <code>RateLimited</code> / 429	<code>RateLimit</code>
<code>LobbyServiceException</code> <code>LobbyNotFound</code> / 404	<code>SessionGone</code>
<code>RequestFailedException</code> 429 / 404 / 5xx / {-1,0}	<code>RateLimit</code> / <code>SessionGone</code> / <code>Transient</code> / <code>Offline</code>
<code>WebException</code> / <code>SocketException</code> / <code>HttpRequestException</code>	<code>Offline</code>
anything else	<code>Unknown</code>

◆ Two design choices worth noting

◆ DESIGN DECISION

Diagnostics log on a strippable channel

NetDiag lines go through `CSDDebug.Log`, which carries `[Conditional("UNITY_EDITOR")] / [Conditional("DEVELOPMENT_BUILD")]`. In a shipping build the compiler removes the call and its argument evaluation — `ClassifyException` and `GetSnapshot` are never invoked, the interpolated string is never built. **Zero runtime cost, zero allocation in release.** It is also runtime-mutable in the editor.

◆ PITFALL / ANTI-PATTERN

A log classifier is not a retry predicate

`ClassifyException` decides what to log;
`PartySessionService.IsTransientSessionException` decides what to retry. They are kept separate on purpose so log format never silently drives retry policy — a `PaymentRequired` class could be interesting to log yet must not trigger a retry. If you change retry behaviour, change the retry predicate, not the log classifier.

`NetworkMonitor` polls reachability every 5 s and emits an explicit transition line (`Online → Offline` (reach=..., t=...)) so a party-flow failure can be cross-referenced against whether the network actually changed around it. Sixteen catch sites across five files carry the NetDiag decoration; the three `NetworkTransitionService` timeouts hard-code `class=Timeout` so a real timeout isn't buried under the `Cancelled` its token would otherwise produce.

PART II · THE RECORD

Design-decisions ledger

The load-bearing decisions, each with what it buys and what it costs. These are locked — the project treats re-opening them as the most common way solved bugs return — so each carries a written rationale that a future change has to argue against explicitly.

Decision	Pros	Cost / trade-off
EAGER per-user Relay ("Always-InParty")	Invites become joins, not create-then-handoff; eliminates the shutdown-and-recreate race class	Every solo player holds a Relay allocation
Two-level sessions (lobby + party)	Cheap global discovery; invites need no host privilege; gameplay transport stays small	Two sessions to keep coherent; the "lobby ≠ party" confusion if conflated
Single-writer SOAP	One place to debug shared state; UI fully decoupled and testable; no two-writer races	A little ceremony to add new state
State machine over boolean flags	Illegal transitions are loud and immediate; one logged lifecycle timeline	Must define every legal transition up front
Session authoritative over presence	Host roster can't be fooled by stale lobby properties (killed B8)	Presence data can be cosmetically stale (harmless)
Live-Instance property for UGS singletons	Never pins <code>null</code> from a constructor that runs before UGS init	Resolves at every use instead of once
Decomposed transition primitives	Each step has its own timeout; recovery is explicit; no "stuck mid-transition"	More methods than one monolithic call
<code>.AsMainThread()</code> boundary helper	Safe SOAP raises and Unity access after every cloud await; one uniform surface	Per-await wrapper discipline (canary-enforced)
Fail-loud (no SOAP null-guards)	Wiring errors surface instantly and obviously	A missing reference crashes rather than degrades
Diagnostics separate from retry policy	Log format never silently drives behaviour; the two can diverge	Two predicates to maintain

◆ A few decisions worth their own note

◆ DESIGN DECISION

One create surface: `EnsurePartySessionAsync`

The four old `RetryCreate*` call sites were not really retries — three were first-time creates and one was recovery. Collapsing them into a single idempotent create-or-no-op removed a whole category of "did we already create one?" ambiguity. Recovery is the only site that clears a stale session first.

◆ DESIGN DECISION**State-machine recovery, not null-checking**

Runtime nulls inside a service method imply an invariant violation, so the response is to **log + drive the state machine to a recoverable state** (typically `Disconnected`), letting the normal sign-in / retry loop pick back up — rather than scattering defensive null-checks that hide the real problem.

◆ KEY INSIGHT**The decisions compose into the invariant**

None of these decisions is exotic on its own. Their value is cumulative: eager sessions keep the state machine small, the state machine keeps recovery honest, single-writer SOAP keeps the UI truthful, `.AsMainThread()` keeps callbacks safe, and authoritative-session keeps two clients agreeing. Together they are what makes “unbreakable under adversarial conditions” a checkable property rather than an aspiration.



APPENDICES

Reference

File and class index, glossary of networking and project terms, and the UGS SDK call map.



APPENDIX A

File & class index

Canonical locations for everything referenced in this document. Line counts are approximate.

◆ Party / presence services — `Assets/_Scripts/Controller/Party/`

File	Lines	Role
<code>HostConnectionService.cs</code>	2034	Orchestrator; single writer to <code>HostConnectionDataSO</code>
<code>PartyInviteController.cs</code>	463	Accept / decline / leave transition sequencing
<code>FriendsInitializer.cs</code>	236	Friends service bridge + presence
<code>Services/PresenceLobbyService.cs</code>	503	Lobby-only session lifecycle
<code>Services/PartySessionService.cs</code>	369	Relay-backed session lifecycle + retry
<code>Services/AcceptanceSignalService.cs</code>	316	Sender ↔ receiver handshake
<code>Services/NetworkTransitionService.cs</code>	288	NM shutdown + connection/scene waits
<code>Services/InviteService.cs</code>	269	Outgoing-invite tracking + serialization
<code>Services/SoapPartyEventBus.cs</code>	206	Centralized SOAP raises
<code>Services/PartyMemberService.cs</code>	188	Member list diffing
<code>Services/LobbyPropertyWriter.cs</code>	182	Mutex + retry property writes
<code>Services/LobbyRefreshScheduler.cs</code>	173	Refresh cadence + boost
<code>StateMachine/PartyStateMachine.cs</code>	155	Validated lifecycle transitions
<code>StateMachine/PartyState.cs</code>	90	The 7 lifecycle states

File	Lines	Role
Interfaces/*	—	IPresenceLobbyService, IPartySessionService, INetworkTransitionService, IPartyMemberService, IInviteService, IPartyStateQuery

◆ Netcode / multiplayer — Assets/_Scripts/Controller/Multiplayer/

File	Lines	Role
ServerPlayerVesselInitializer.cs	472	Base server vessel spawner
MultiplayerSetup.cs	462	NetworkManager lifecycle + UGS sessions
ClientPlayerVesselInitializer.cs	395	Client pair init + RPCs
ServerPlayerVesselInitializerWithAI.cs	353	AI pre-spawn + team balancing
MenuCrystalClickHandler.cs	347	Play-from-menu freestyle toggle
MenuVesselSelectionPanelController.cs	251	Network-aware vessel swap
ArcadeConfigSyncManager.cs	243	Config sync
MenuServerPlayerVesselInitializer.cs	238	Menu autopilot spawner
DomainAssigner.cs	—	Team pool assignment

◆ Cross-cutting & SOAP

File	Role
Assets/_Scripts/System/AuthenticationServiceFacade.cs	Auth single-writer
Assets/_Scripts/System/FriendsServiceFacade.cs	Friends single-writer
Assets/_Scripts/Controller/Player/Player.cs	NetworkBehaviour + 6 NetworkVariables
Assets/_Scripts/Utility/MainThreadDispatcher.cs	SyncContext-based main-thread switch
Assets/_Scripts/Utility/ClassExtensions/UniTaskExtensions.cs	.AsMainThread() overloads
Assets/_Scripts/Utility/NetworkDiagnostics.cs	NetDiag classifier
Assets/_Scripts/Utility/DataContainers/HostConnectionDataS0.cs	Party SOAP container
Assets/_Scripts/Utility/DataContainers/FriendsDataS0.cs	Friends SOAP container

◆ Canonical engineering docs — Docs/

PartySystem/ (ARCHITECTURE · REFACTOR · BUGS · TESTS · TODOS · MPPM_SESSION_LOG) · PresenceSystem/ (ARCHITECTURE · REFACTOR · BUGS · TESTS · TODOS) · NetworkDiagnostics/ (README · TESTS · TODOS) · THREADING.md · SCENES.md. This document is synthesised from those sources.

APPENDIX B

Glossary

Term	Meaning
UGS	Unity Gaming Services — the cloud suite (Auth, Sessions, Lobby, Relay, Friends, ...).
Session (UGS)	A managed group of players. With Relay it is a party session; without, a lobby-only presence lobby.
Lobby	The UGS service backing the presence session — player roster + per-player properties.
Relay	UGS service that allocates a relay server so NAT-bound clients connect without port-forwarding.
Presence Lobby	Lobby-only session every signed-in player joins for discovery and invite exchange (no Relay).
Party Session	Relay-backed session that carries gameplay networking for the people flying together.
Netcode (NGO)	Unity Netcode for GameObjects — the replication framework.
NetworkManager	The Netcode singleton driving connection, scene sync, and object spawning.
NetworkBehaviour	A component that can own <code>NetworkVariable</code> s and RPCs (e.g. <code>Player</code> , vessel status).
NetworkVariable	A replicated field with a defined writer (owner or server).
ClientRpc / ServerRpc	A remote call from server → clients / client → server.
NetworkObject	A spawned, replicated <code>GameObject</code> identified by <code>NetworkObjectId</code> .
Host	A player that is server and client simultaneously.
Eager / “Always-InParty”	Every player creates their own Relay party session on entering the menu.
Presence (Friends)	Rich status a friend sees — “In Menu”, “In Party”, “In Game” + scene/vessel/session.
SOAP	Scriptable Object Architecture Pattern — <code>ScriptableVariable</code> state + <code>ScriptableEvent</code> channels.
Single-writer	A piece of SOAP state has exactly one writer; everyone else reads.
UniTask	Allocation-light async/await for Unity (<code>com.cysharp.unitask</code>).
<code>.AsMainThread()</code>	The boundary helper that re-asserts Unity's main thread after a cross-thread await.
MainThreadDispatcher	Captures Unity's <code>SynchronizationContext</code> and switches onto it reliably.
MPPM	Multiplayer Play Mode — Unity feature running several virtual players in one editor.
NetDiag	The diagnostic one-liner appended to party/lobby/session catch blocks.
CSDebug	Project logger that strips from release builds and is runtime-mutable.
Domain	Team / affiliation identity (Jade, Ruby, Gold; Blue = neutral / no-team).
Vessel	A player- or AI-controlled ship; a <code>NetworkBehaviour</code> spawned by the server.

Term	Meaning
DI (Reflex)	Dependency injection; <code>AppManager</code> is the root installer.
Lazy DI singleton	A service constructed on first injection — possibly before UGS init completes.

APPENDIX C

UGS SDK call map

Which service method maps to which Unity Gaming Services SDK call. This is the surface the whole party/presence/friends stack is built on.

◆ Sessions & Relay

Layer · method	UGS SDK call
<code>PresenceLobbyService.JoinOrCreateAsync</code>	<code>QuerySessionsAsync</code> (filter <code>PRESENCE_LOBBY</code>) → <code>CreateSessionAsync</code>
<code>PresenceLobbyService.ConvergeToCanonicalAsync</code>	<code>QuerySessionsAsync</code> → <code>JoinSessionByIdAsync</code>
<code>PresenceLobbyService.RefreshAsync</code>	<code>ISession.RefreshAsync</code>
<code>PresenceLobbyService.LeaveAsync</code>	<code>ISession.AsHost().DeleteAsync</code> / <code>LeaveAsync</code>
<code>PartySessionService.CreateAsync</code>	<code>CreateSessionAsync(SessionOptions.WithRelayNetwork())</code>
<code>PartySessionService.JoinByIdAsync</code>	<code>JoinSessionByIdAsync</code> (with retry)
<code>PartySessionService.RefreshAsync</code>	<code>ISession.RefreshAsync</code>
<code>PartySessionService.LeaveAsync</code>	<code>ISession.AsHost().DeleteAsync</code> / <code>LeaveAsync</code>
<code>HostConnectionService.KickPartyMemberAsync</code>	<code>ISession.AsHost().RemovePlayerAsync</code>

◆ Lobby properties (the invite channel)

Layer · method	UGS SDK call
<code>LobbyPropertyWriter.SaveWithRetryAsync</code>	set player properties + <code>ISession.RefreshAsync</code> (mutex + retry)
<code>HostConnectionService.SendInviteAsync</code>	write <code>invite_target</code> / <code>invite_data</code> player properties
<code>HostConnectionService</code> presence scan	read each lobby player's properties each refresh tick

◆ Authentication

Layer · method	UGS SDK call
<code>AuthenticationServiceFacade.EnsureSignedInAnonymouslyAsync</code>	<code>AuthenticationService.SignInAnonymouslyAsync</code>
<code>AuthenticationServiceFacade.TrySignInCachedAsync</code>	cached-session restore
<code>AppManager</code> startup	<code>UnityServices.InitializeAsync</code>

◆ Friends

Layer · method	UGS SDK call
<code>FriendsServiceFacade.InitializeAsync</code>	<code>FriendsService.InitializeAsync</code>

Layer · method	UGS SDK call
SendFriendRequestByNameAsync / SendFriendRequestAsync	AddFriendByNameAsync / AddFriendAsync
AcceptFriendRequestAsync	AddFriendAsync
DeclineFriendRequestAsync / CancelFriendRequestAsync	DeleteIncomingFriendRequestAsync / DeleteOutgoingFriendRequestAsync
RemoveFriendAsync	DeleteFriendAsync
BlockPlayerAsync / UnblockPlayerAsync	AddBlockAsync / DeleteBlockAsync
SetPresenceAsync / SetAvailabilityAsync	SetPresenceAsync / SetPresenceAvailabilityAsync
RefreshAsync	ForceRelationshipsRefreshAsync

◆ KEY INSIGHT

Every one of these awaits `.AsMainThread()`

Without exception, each SDK call above is awaited through the `.AsMainThread()` boundary helper so its continuation — and any SOAP event it raises — resumes on Unity's main thread. That single, uniform discipline is what keeps the entire cloud surface from leaking ThreadPool continuations into Unity state. It is the connective tissue under every row in this table.

End of document. Synthesised from the canonical engineering docs under `Docs/` and the party, multiplayer, and player source under `Assets/_Scripts/`. — Cosmic Shore · Froglet Inc.